

6. *Product of Floating Point Numbers:* If $\alpha_i, i = 1, 2, 3, \dots, n$ are n floating point numbers, then

$$\text{fl} \left(\prod_{i=1}^n \alpha_i \right) = \left(\prod_{i=1}^n \alpha_i \right) (1 + \delta)^n$$

with $|\delta| < \frac{1}{2}b^{1-p}$. Consequently, rounding errors in floating point products create only minute relative errors.

7. [Ste98], [Wil65] *Dot (or Inner) Product of Floating Point Numbers:* Let $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{y} \in \mathbb{R}^n$ be vectors of floating point numbers and let s be the finite precision dot product $s = \text{fl}(\mathbf{x}^T \mathbf{y}) = x_1 \otimes y_1 \oplus x_2 \otimes y_2 \oplus x_3 \otimes y_3 \oplus \dots \oplus x_n \otimes y_n$ evaluated in the order shown, multiplications first followed by additions from left to right. If $n < 0.1/(\frac{1}{2}b^{1-p})$ and neither overflow nor any kind of underflow occurs, then the following hold.

(a) $s = \text{fl}(\mathbf{x}^T \mathbf{y}) = x_1 y_1 (1 + \delta_1) + x_2 y_2 (1 + \delta_2) + x_3 y_3 (1 + \delta_3) + \dots + x_n y_n (1 + \delta_n)$ with $|\delta_1| < 1.06n(\frac{1}{2}b^{1-p})$ and $|\delta_j| < 1.06(n - j + 2)(\frac{1}{2}b^{1-p}) \leq 1.06n(\frac{1}{2}b^{1-p})$, for $j = 2, 3, 4, \dots, n$.

(b) $s = \text{fl}(\mathbf{x}^T \mathbf{y}) = \hat{\mathbf{x}}^T \hat{\mathbf{y}}$ for some vectors $\hat{\mathbf{x}}, \hat{\mathbf{y}} \in \mathbb{R}^n$ satisfying $|\mathbf{x} - \hat{\mathbf{x}}| \leq |\mathbf{x}|(1 + 1.06n(\frac{1}{2}b^{1-p}))$ and $|\mathbf{y} - \hat{\mathbf{y}}| \leq |\mathbf{y}|(1 + 1.06n(\frac{1}{2}b^{1-p}))$. So, s is the mathematically correct product of the vectors $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$ each of whose entries differ from the corresponding entries of $\mathbf{x}$ or $\mathbf{y}$ by minute relative errors.

There are infinitely many choices of $\hat{\mathbf{x}}$ and $\hat{\mathbf{y}}$. In the notation of Fact 7(a), two are $\hat{x}_j = x_j(1 + \delta_j)$, $\hat{y}_j = y_j$ and $\hat{x}_j = x_j(1 + \delta_j)^{1/2}$, $\hat{y}_j = y_j(1 + \delta_j)^{1/2}$, $j = 1, 2, 3, \dots, n$.

(c) If $\mathbf{x}^T \mathbf{y} \neq 0$, then the relative error in s is bounded as

$$\left| \frac{s - \mathbf{x}^T \mathbf{y}}{\mathbf{x}^T \mathbf{y}} \right| \leq 1.06n(\frac{1}{2}b^{1-p}) \frac{|\mathbf{x}|^T |\mathbf{y}|}{|\mathbf{x}^T \mathbf{y}|}.$$

The bound shows that if there is little cancellation in the sum, then s has small relative error. The bound allows the possibility that s has large relative error when there is substantial cancellation in the sum. Indeed this is often the case.

8. *Rounding Error Bounds for Floating Point Matrix Operations:* In the following, A and B are matrices each of whose entries is a floating point number, $\mathbf{x}$ is a vector of floating point numbers, and c is a floating point number. The matrices A and B are compatible for matrix addition or matrix multiplication as necessary and E is an error matrix (usually different in each case) whose entries may or may not be floating point numbers. The integer dimension n is assumed to satisfy $n < 0.1/(\frac{1}{2}b^{1-p})$.

If neither overflows nor any kind of underflow occurs, then

(a) $\text{fl}(cA) = cA + E$ with $|E| \leq \frac{1}{2}b^{1-p}|cA|$.

(b) $\text{fl}(A + B) = (A + B) + E$ with $|E| \leq \frac{1}{2}b^{1-p}|A + B|$.

(c) If $\mathbf{x} \in \mathbb{R}^n$ and $A \in \mathbb{R}^{m \times n}$, then $\text{fl}(A\mathbf{x}) = (A + E)\mathbf{x}$ with $|E| \leq 1.06n(\frac{1}{2}b^{1-p})$.

(d) *Matrix multiplication:* If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times q}$, $\text{fl}(AB) = AB + E$ with $|E| \leq 1.06n(\frac{1}{2}b^{1-p})|A||B|$.

Note that if $|AB| \approx |A||B|$, then each entry in $\text{fl}(AB)$ is correct to within a minute relative error. Otherwise, subtractive cancellation is possible and some entries of $\text{fl}(AB)$ may have large relative errors.

(e) Let $\|\cdot\|$ be a matrix norm satisfying $\|E\| \leq \| |E| \|$. (All of $\|\cdot\|_1, \|\cdot\|_2, \|\cdot\|_p, \|\cdot\|_\infty$ and $\|\cdot\|_F$ satisfy this requirement.) If $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times q}$, then $\text{fl}(AB) = AB + E$ with $\|E\| \leq 1.06n(\frac{1}{2}b^{1-p})\|A\|\|B\|$.

(f) *Matrix multiplication by an orthogonal matrix:* If $Q \in \mathbb{R}^{n \times n}$ is an orthogonal matrix, i.e., if $Q^T = Q^{-1}$, and if $A \in \mathbb{R}^{m \times n}$, then

$$(i) \text{ fl}(QA) = QA + E \text{ with } \|E\|_2 \leq 1.06n^2 \left(\frac{1}{2}b^{1-p}\right) \|A\|_2 \text{ and } \|E\|_F \leq 1.06n^{3/2} \left(\frac{1}{2}b^{1-p}\right) \|A\|_F.$$

$$(ii) \text{ fl}(QA) = Q(A + \hat{E}) \text{ with } \|\hat{E}\|_2 \leq 1.06n^2 \left(\frac{1}{2}b^{1-p}\right) \|A\|_2 \text{ and } \|\hat{E}\|_F \leq 1.06n^{3/2} \left(\frac{1}{2}b^{1-p}\right) \|A\|_F.$$

Note that this bound shows that if one of the factors is an orthogonal matrix, then subtractive cancellation in floating point matrix multiplication is limited to those entries (if any) that have magnitude substantially smaller than the other, possibly nonorthogonal factor. Many particularly successful numerical methods derive their robustness in the presence of rounding errors from this observation. (See, for example, [Dat95], [GV96], [Ste98], or [Wil65].)

Examples:

- The following table lists typical floating point systems in common use at the time of this writing.

	radix b	precision p	$e_{\min}$	$e_{\max}$
Some calculators	10	14	-99	99
Some calculators	10	14	-999	999
IEEE 754r decimal 64 [IEEE754r]	10	16	-383	384
IEEE 754r decimal 128 [IEEE754r]	10	34	-6143	6144
IEEE 754r binary 32 (Single) [IEEE754r]	2	24	-126	127
IEEE 754r binary 64 (Double) [IEEE754r]	2	53	-1022	1023
IEEE 754r binary 128 (Double Extended) [IEEE754r]	2	113	-16382	16383

- Consider $p = 5$ digit, radix $b = 10$ floating-point arithmetic with round-to-nearest, ties-to-even rounding mode.
 - Floating point addition is not associative:* If $\alpha = 1$, $\beta = 10^5$, and $\gamma = -10^5$, then $(\alpha \oplus \beta) \oplus \gamma = 0$ but $\alpha \oplus (\beta \oplus \gamma) = 1$. (This is also an example of subtractive cancellation.)
 - Floating point multiplication/division is not associative:* If $\alpha = 3$, $\beta = 1$, and $\gamma = 3$, then $\alpha \otimes (\beta \oslash \gamma) = .99999$ but $(\alpha \otimes \beta) \oslash \gamma = 1$. If $\alpha = 44444$, $\beta = 55555$ and $\gamma = 66666$, then $\alpha \otimes (\beta \otimes \gamma) = 1.6460 \times 10^{14}$ but $(\alpha \otimes \beta) \otimes \gamma = 1.6461 \times 10^{14}$. Although different, both expressions have minute relative error. It is generally the case that floating point products have small relative errors. (See Fact 6.)
 - Floating point multiplication does not distribute across floating point addition:* If $\alpha = 9$, $\beta = 1$, and $\gamma = -.99999$, then $\alpha \otimes (\beta \oplus \gamma) = 9.0000 \times 10^{-5}$ but $(\alpha \otimes \beta) \oplus (\alpha \otimes \gamma) = 1.0000 \times 10^{-4}$.
- Subtractive Cancellation:* In $p = 5$, radix $b = 10$ arithmetic with round-to-nearest, ties-to-even rounding, the expression $(\sqrt{1 + 10^{-4}} - 1)/10^{-4}$ evaluates as follows. (Here we assume that the floating point evaluation of the square root gives the same result as rounding the exact square root to $p = 5$, radix $b = 10$ digits.)

$$\begin{aligned}
(\text{fl}(\sqrt{1.0000 \oplus 10^{-4}}) \ominus 1) \oslash 10^{-4} &= (\text{fl}(\sqrt{1.0001}) \ominus 1.0000) \oslash 10^{-4} \\
&= (\text{fl}(1.000049999 \dots) \ominus 1.0000) \oslash 10^{-4} \\
&= (1.0000 \ominus 1.0000) \oslash 10^{-4} \\
&= 0.0000.
\end{aligned}$$

In exact, infinite precision arithmetic, $(\sqrt{1 + 10^{-4}} - 1)/10^{-4} = .4999875 \dots$, so the relative error is 1. Note that zero rounding error occurs in the subtraction. The only nonzero rounding error is the square root, which has minute relative error roughly 5×10^{-5} , but this is enough to ruin the final

result. Subtractive cancellation did not cause the large relative error. It only exposed the unfortunate fact that the result was ruined by earlier rounding errors.

4. *Relative vs. Absolute Errors:* Consider $\hat{x}_1 = 31416$ as an approximation to $x_1 = 10000\pi$ and $\hat{x}_2 = 3.07$ as an approximation to $x_2 = \pi$. The absolute errors are nearly the same: $|\hat{x}_1 - x_1| \approx -0.0735$ and $|\hat{x}_2 - x_2| \approx -0.0716$. On the other hand, the relative error in the first case, $\frac{|\hat{x}_1 - x_1|}{|x_1|} \approx 2 \times 10^{-6}$, is much smaller than the relative error in the second case, $\frac{|\hat{x}_2 - x_2|}{|x_2|} \approx 2 \times 10^{-2}$. The smaller relative error shows that $\hat{x}_1 = 31416$ is a better approximation to 10000π than $\hat{x}_2 = 3.07$ is to π . The absolute errors gave no indication of this.

37.7 Algorithms and Efficiency

In this section, we introduce efficiency of algorithms.

Definitions:

An **algorithm** is a precise set of instructions to perform a task. The algorithms discussed here perform mathematical tasks that transform an initial data set called the **input** into a desired result final data set called the **output** using an ordered list of arithmetic operations, comparisons, and decisions. For example, the Gaussian elimination algorithm (see Chapter 39.3) solves the linear system of equations $A\mathbf{x} = \mathbf{b}$ by transforming the given, nonsingular matrix $A \in \mathbb{R}^{n \times n}$ and right-hand-side $\mathbf{b} \in \mathbb{R}^n$ into a vector $\mathbf{x}$ satisfying $A\mathbf{x} = \mathbf{b}$.

One algorithm is more **efficient** than another if it accomplishes the same task with a lower cost of computation. Cost of computation is usually dominated by the direct and indirection cost of execution time. So, in general, in a given computational environment, the more efficient algorithm finishes its task sooner. (The very real economic cost of algorithm development and implementation is not a part of the efficiency of an algorithm.) However, the amount of primary and secondary memory required by an algorithm or the expense of and availability of the necessary equipment to execute the algorithm may also be significant part of the cost. In this sense, an algorithm that can accomplish its task on an inexpensive, programmable calculator is more efficient than one that needs a supercomputer.

For this discussion, a floating point operation or **flop** consists of a floating point addition, subtraction, multiplication, division, or square root along with any necessary subscripting and loop index overhead. In Fortran $A(I, J) = A(I, J) + C * A(K, J)$ performs two flops. (Note that this is a slightly different definition of flop than is used in computer engineering.)

Formal algorithms are often specified in terms of an informal computer program called **pseudo-code**.

On early digital computers, computation time was heavily dominated by evaluating floating point operations. So, traditionally, numerical analysts compare the efficiency of two algorithms by counting the number of floating point operations each of them executes. If n measures the input data set size, e.g., an input matrix $A \in \mathbb{R}^{n \times n}$, then an $O(n^p)$ algorithm is one that, for some positive constant c , performs cn^p plus a sum of lower powers of n floating point operations.

Facts:

For proofs and additional background, see, for example, [Dat95], [GV96], [Ste98], or [Wil65].

In choosing a numerical method, efficiency must be balanced against considerations like robustness against rounding error and likelihood of failure.

Despite tradition, execution time has never been more than roughly proportional to the amount of floating point arithmetic. On modern computers with fast floating point arithmetic, multiple levels of cache memory, overlapped instruction execution, and parallel processing, execution time is correlated more closely with the number of cache misses (i.e., references to main RAM memory) than it is to the number of floating point operations. In addition, the relative execution time of algorithms depends strongly on the environment in which they are executed.

Nevertheless, for algorithms highly dominated by floating point arithmetic, flop counts are still useful. Despite the complexities of modern computers, flop counts typically expose the rate at which execution time increases as the size of the problem increases. Solving linear equations $\mathbf{Ax} = \mathbf{b}$ for given $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$ by Gaussian elimination with partial pivoting is an $O(n^3)$ algorithm. For larger values of n , solving a $2n$ -by- $2n$ system of equations takes roughly eight times longer than an n -by- n system.

1. Triangular back substitution is an $O(n^2)$ algorithm to solve a triangular system of linear equations $T\mathbf{x} = \mathbf{b}$, $\mathbf{b} \in \mathbb{R}^n$ and $T \in \mathbb{R}^{n \times n}$, with $t_{ij} = 0$ whenever $i > j$ [GV96]. (See the pseudo-code algorithm below.)
2. [GV96] Gaussian elimination with partial pivoting (cf. Algorithm 1, Section 38.3) is an $O(n^3)$ algorithm to solve a system of equations $\mathbf{Ax} = \mathbf{b}$ with $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$.
3. Because of the need to repeatedly search an entire submatrix, Gaussian elimination with complete pivoting is an $O(n^4)$ algorithm to solve a system of equations $\mathbf{Ax} = \mathbf{b}$ with $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$ [GV96]. Hence, complete pivoting is not competitive with $O(n^3)$ methods like Gaussian elimination with partial pivoting.
4. [GV96] The QR-factorization by Householder's method is an $O(n^3)$ algorithm to solve a system of equations $\mathbf{Ax} = \mathbf{b}$ with $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$.
5. [GV96] The QR factorization by Householder's method to solve the least squares problem $\min \|\mathbf{Ax} - \mathbf{b}\|_2$ for given $A \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$ is an $O(n^2(m - n/3))$ algorithm.
6. [GV96] The singular value decomposition using the Golub–Kahan–Reinsch algorithm to solve the least squares problem $\min \|\mathbf{Ax} - \mathbf{b}\|_2$ for given $A \in \mathbb{R}^{m \times n}$ and $\mathbf{b} \in \mathbb{R}^m$ is an $O(m^2n + mn^2 + n^3)$ algorithm.
7. [GV96] The implicit, double-shift QR iteration algorithm to find all eigenvalues of a given matrix $A \in \mathbb{R}^{n \times n}$ is an $O(n^3)$ algorithm.
8. Cramer's rule for solving the system of equations $\mathbf{Ax} = \mathbf{b}$ for given $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$ in which determinants are evaluated using minors and cofactors is an $O((n + 1)n!)$ algorithm and is impractical for all but small values of n .
9. Cramer's rule for solving the system of equations $\mathbf{Ax} = \mathbf{b}$ for given $A \in \mathbb{R}^{n \times n}$ and $\mathbf{b} \in \mathbb{R}^n$ in which determinants are evaluated using Gaussian elimination is an $O(n^4)$ algorithm and is not competitive with $O(n^3)$ methods like Gaussian elimination with partial pivoting.

Examples:

1. It takes roughly 4.6 seconds on a 2GHz Pentium workstation, using Gaussian elimination with partial pivoting, to solve the $n = 2000$ linear equations in 2000 unknowns $\mathbf{Ax} = \mathbf{b}$ in which the entries of A and $\mathbf{b}$ are normally distributed pseudo-random numbers with mean zero and variance one. It takes roughly 34 seconds to solve a similar $n = 4000$ system of equations. This is consistent with the estimate that Gaussian elimination with partial pivoting is an $O(n^3)$ algorithm.
2. This is an example of a formal algorithm specified in pseudo-code. Consider the problem of solving for $\mathbf{y}$ in the upper triangular system of equations $T\mathbf{y} = \mathbf{b}$, where $\mathbf{b} \in \mathbb{R}^n$ is a given right-hand-side vector and $T \in \mathbb{R}^{n \times n}$ is a given nonsingular upper triangular matrix; i.e., $t_{ij} = 0$ for $i > j$ and $t_{ii} \neq 0$ for $i = 1, 2, 3, \dots, n$.

Input: A nonsingular, upper triangular matrix $T \in \mathbb{R}^{n \times n}$ and a vector $\mathbf{b} \in \mathbb{R}^n$.

Output: The vector $\mathbf{y} \in \mathbb{R}^n$ satisfying $T\mathbf{y} = \mathbf{b}$.

Step 1. $y_n \leftarrow b_n / t_{nn}$

Step 2. For $i = n - 1, n - 2, \dots, 2, 1$ do

$$2.1 \quad s_i \leftarrow b_i - \sum_{j=i+1}^n t_{ij} y_j$$

$$2.2 \quad y_i = s_i / t_{ii}$$

37.8 Numerical Stability and Instability

Numerically stable algorithms, despite rounding and truncation errors, produce results that are roughly as accurate as the errors in the input data allow. *Numerically unstable* algorithms allow rounding and truncation errors to produce results that are substantially less accurate than the errors in the input data allow. This section concerns numerical stability and instability and is loosely based on [Bun87].

Definitions:

A computational problem is the task of evaluating a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ at a particular data point $\mathbf{x} \in \mathbb{R}^n$. A numerical algorithm that is subject to rounding and truncation errors evaluates a perturbed function $\hat{f}(\mathbf{x})$. Throughout this section ε represents a modest multiple of the unit round.

The **forward error** is $f(\mathbf{x}) - \hat{f}(\mathbf{x})$, the difference between the mathematically exact function evaluation and the perturbed function evaluation.

The **backward error** is a vector $\mathbf{e} \in \mathbb{R}^n$ of smallest norm for which $f(\mathbf{x} + \mathbf{e}) = \hat{f}(\mathbf{x})$. If no such $\mathbf{e}$ exists, then the backward error is undefined. This definition is illustrated in Figure 37.1 [Ste98, p. 123].

An algorithm is **forward stable** if, despite rounding and truncation errors,

$$\frac{\|f(\mathbf{x}) - \hat{f}(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \varepsilon$$

for all valid input data $\mathbf{x}$. In a forward stable algorithm, the forward relative error is small for all valid input data $\mathbf{x}$ despite rounding and truncation errors in the algorithm.

An algorithm is **backward stable** or **strongly stable** if the backward relative error $\mathbf{e}$ exists and satisfies the relative error bound $\|\mathbf{e}\| \leq \varepsilon \|\mathbf{x}\|$ for all valid input data $\mathbf{x}$ despite rounding and truncation errors.

In this context, “small” means a modest multiple of the size of the errors in the data $\mathbf{x}$. If rounding errors are the only relevant errors, then “small” means a modest multiple of the unit round.

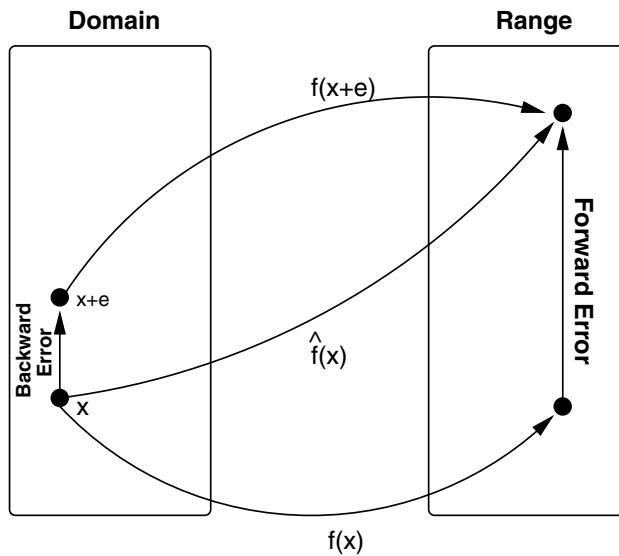


FIGURE 37.1 A computational problem is the task of evaluating a function f at a particular data point $\mathbf{x}$. A numerical algorithm that is subject to rounding and truncation errors evaluates a perturbed function $\hat{f}(\mathbf{x})$. The **forward error** is $f(\mathbf{x}) - \hat{f}(\mathbf{x})$, the difference between the mathematically exact function evaluation and the perturbed function evaluation. The **backward error** is a vector $\mathbf{e} \in \mathbb{R}^n$ of smallest norm for which $f(\mathbf{x} + \mathbf{e}) = \hat{f}(\mathbf{x})$ [Ste98, p. 123]. If no such vector $\mathbf{e}$ exists, then there is no backward error.

A finite precision numerical algorithm is **weakly numerically stable** if rounding and truncation errors cause it to evaluate a perturbed function $\hat{f}(\mathbf{x})$ satisfying the relative error bound

$$\frac{\|f(\mathbf{x}) - \hat{f}(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \varepsilon \text{cond}_f(\mathbf{x})$$

for all valid input data $\mathbf{x}$. In a weakly stable algorithm, the *magnitude* of the forward error is no greater than magnitude of an error that could be induced by perturbing the data by small multiple of the unit round. Note that this does *not* imply that there is a small backward error or even that a backward error exists. (An even weaker kind of “weak stability” requires the relative error bound only when $\mathbf{x}$ is well-conditioned [Bun87].)

An algorithm is **numerically stable** if rounding errors and truncation errors cause it to evaluate a perturbed function $\hat{f}(\mathbf{x})$ satisfying

$$\frac{\|f(\mathbf{x} + \mathbf{e}) - \hat{f}(\mathbf{x})\|}{\|f(\mathbf{x})\|} \leq \varepsilon$$

for some small relative-to- $\|\mathbf{x}\|$ backward error $\mathbf{e}$, $\|\mathbf{e}\| \leq \varepsilon \|\mathbf{x}\|$. In a numerically stable algorithm, $\hat{f}(\mathbf{x})$ lies near a function with small backward error.

Figure 37.2 illustrates the definitions of stability. The black dot on the left represents a nominal data point $\mathbf{x}$. The black dot on the right is the exact, unperturbed value of $f(\mathbf{x})$. The shaded region on the left represents the small relative-to- $\|\mathbf{x}\|$ perturbations of $\mathbf{x}$. The shaded region on the right is its exact image under $f(\mathbf{x})$.

In a weakly stable numerical method, the computed function value $\hat{f}(\mathbf{x})$ lies inside the large circle with radius equal to the longest distance from the black dot $f(\mathbf{x})$ to the furthest point in the shaded region containing it. The error in a weakly stable algorithm is no larger than would have been obtained from a backward stable algorithm. However, the actual result may or may not correspond to a small perturbation of the data.

In a numerically stable algorithm, $\hat{f}(\mathbf{x})$ lies either near or inside the shaded region on the right.

In a backward stable algorithm, $\hat{f}(\mathbf{x})$ lies in the shaded region, but, if the data are ill-conditioned as in the illustration, $\hat{f}(\mathbf{x})$ may have a large relative error. (To avoid clutter, there is no arrow illustrating a backward stable algorithm.)

In a forward stable algorithm, $\hat{f}(\mathbf{x})$ has a small relative error, but $\hat{f}(\mathbf{x})$ may or may not correspond to a small perturbation of the data.

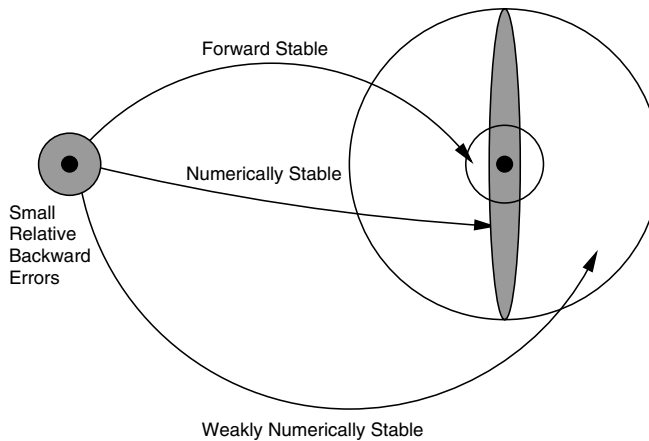


FIGURE 37.2 The black dot on the left represents a nominal data point $\mathbf{x}$. The black dot on the right is the exact, unperturbed value of $f(\mathbf{x})$. The shaded region on the left represents the small relative-to- $\|\mathbf{x}\|$ perturbations of $\mathbf{x}$. The shaded region on the right is its exact image under $f(\mathbf{x})$. The diagram illustrates the error behavior of a weakly numerically stable algorithm, a numerically stable algorithm, and a forward stable algorithm.

Facts:

1. A numerically stable algorithm applied to an ill-conditioned problem may produce inaccurate results. For ill-conditioned problems even small errors in the input data may lead to large errors in the computed solution. Just as no numerical algorithm can reliably correct errors in the data or create information not originally implicit in the data, nor can a numerical algorithm reliably calculate accurate solutions to ill-conditioned problems.
2. Rounding and truncation errors in a backward stable algorithm are equivalent to a further perturbation of the data. The computed results of a backward stable algorithm are realistic in the sense that they are what would have been obtained in exact arithmetic from an extra rounding-error-small relative perturbation of the data. Typically this extra error is negligible compared to other errors already present in the data.
3. The forward error that occurs in a backward stable algorithm obeys the asymptotic condition number bound in Fact 2 of Section 37.4. **Backward error analysis** is based on this observation.
4. [Dat95], [GV96] *Some Well-Known Backward Stable Algorithms*:
 - (a) Fact 4 in Section 37.6 implies that a single floating point operation is both forward and backward stable.
 - (b) Fact 7b in section 37.6 shows that the given naive dot product algorithm is backward stable. The algorithm is not, in general, forward stable because there may be cancellation of significant digits in the summation.
 - (c) *Gaussian elimination*: Gaussian elimination with complete pivoting is backward stable. Gaussian elimination with partial pivoting is not, strictly speaking, backward stable. However, linear equations for which the algorithm exhibits instability are so extraordinarily rare that the algorithm is said to be “backward stable in practice” [Dat95, GV96].
 - (d) *Triangular back substitution*: The back-substitution algorithm in Example 2 in Section 37.7 is backward stable. It can be shown that the rounding error corrupted computed solution $\hat{\mathbf{x}}$ satisfies $(T + E)\hat{\mathbf{x}} = \mathbf{b}$, where $|e_{ij}| \leq \epsilon |t_{ij}|$, $i, j = 1, 2, 3, \dots, n$. Thus, the computed solution $\hat{\mathbf{x}}$ solves a nearby system. The back-substitution process is, therefore, backward stable.
 - (e) *QR factorization*: The Householder and Givens methods for factorization of $A = QR$, where Q is orthogonal and R is upper triangular, are backward stable.
 - (f) *SVD computation*: The Golub–Kahan–Reinsch algorithm is a backward stable algorithm for finding the singular value decomposition $A = U\Sigma V^T$, where Σ is diagonal and U and V are orthogonal.
 - (g) *Least-square problem*: The Householder QR factorization, the Givens QR factorization, and Singular Value Decomposition (SVD) methods for solving linear least squares problems are backward stable.
 - (h) *Eigenvalue computations*: The implicit double-shift QR iteration is backward stable.

Examples:

1. An example of an algorithm that is forward stable but not backward stable is the natural computation of the outer product $A = \mathbf{xy}^T$ from vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$: for $i, j = 1, 2, 3, \dots, n$, set $a_{ij} \leftarrow x_i \otimes y_j$. This algorithm produces the correctly rounded value of the exact outer product, so it is forward stable. However, in general, rounding errors perturb the rank 1 matrix $\mathbf{xy}^T$ into a matrix of higher rank. So, the rounding error perturbed outer product is not equal to the outer product of any pair of vectors; i.e., there is no backward error, so the algorithm is not backward stable.
2. *Backward vs. Forward Errors*: Consider the problem of evaluating $f(x) = e^x$ at $x = 1$. One numerical method is to sum several terms of the Taylor series for e^x . If $f(x)$ is approximated by the truncated Taylor series $\hat{f}(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{3!}$, then $f(1) = e \approx 2.7183$ and $\hat{f}(1) \approx 2.6667$.

The forward error is $f(1) - \hat{f}(1) \approx 2.7183 - 2.6667 = 0.0516$. The backward error is $1 - y$, where $f(y) = \hat{f}(1)$, i.e., the backward error is $1 - \ln(\hat{f}(1)) \approx .0192$.

3. *A Numerically Unstable Algorithm:* Consider the computational problem of evaluating the function $f(x) = \ln(1 + x)$ for x near zero. The naive approach is to use $\text{fl}(\ln(1 \oplus x))$, i.e., add one to x in finite precision arithmetic and evaluate the natural logarithm of the result also in finite precision arithmetic. (For this discussion, assume that if z is a floating point number, then $\ln(z)$ returns the correctly rounded exact value of $\ln(z)$.) Applying this very simple algorithm to $x = 10^{-16}$ in $p = 16$, radix $b = 10$ arithmetic, gives $\text{fl}(\ln(1 \oplus 10^{-16})) = \text{fl}(\ln(1)) = 0$. The exact value of $\ln(1 + 10^{-16}) \approx 10^{-16}$, so the rounding error corrupted result has relative error 1. It is 100% incorrect.

However, the function $f(x) = \ln(1 + x)$ is well-conditioned when x is near zero. Moreover, $\lim_{x \rightarrow 0} \text{cond}_f(x) = 1$. The large relative error is not due to ill-conditioning. It demonstrates that this simple algorithm is numerically unstable for x near zero.

4. An alternative algorithm to evaluate $f(x) = \ln(1 + x)$ that does not suffer the above gross numerical instability for x near zero is to sum several terms of the Taylor series

$$\ln(1 + x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots.$$

Although it is adequate for many purposes, this method can be improved. Note also that the series does not converge for $|x| > 1$ and converges slowly if $|x| \approx 1$, so some other method (perhaps $\text{fl}(\ln(1 \oplus x))$) is needed when x is not near zero.

5. *Gaussian Elimination without Pivoting:* Gaussian elimination without pivoting is not numerically stable. For example, consider solving a system of two equations in two unknowns

$$\begin{aligned} 10^{-10}x_1 + x_2 &= 1 \\ x_1 + 2x_2 &= 3 \end{aligned}$$

using $p = 9$ digit, radix $b = 10$ arithmetic. Eliminating x_2 from the second equation, we obtain

$$\begin{aligned} 10^{-10}x_1 + x_2 &= 1 \\ (2 \ominus 10^{10})x_2 &= 3 \ominus 10^{10}, \end{aligned}$$

which becomes

$$\begin{aligned} 10^{-10}x_1 + x_2 &= 1 \\ -10^{10}x_2 &= -10^{10}, \end{aligned}$$

giving $x_2 = 1$, $x_1 = 0$. The exact solution is $x_1 = (1 - 2 \times 10^{-10}) \approx 1$, $x_2 = (1 - 3 \times 10^{-10})/1 - 2 \times 10^{-10} \approx 1$.

The ∞ -norm condition number of the coefficient matrix $A = \begin{bmatrix} 10^{-10} & 1 \\ 1 & 2 \end{bmatrix}$ is $\kappa(A) \approx 9$, so the

large error in the rounding error corrupted solution is not due to ill-conditioning. Hence, Gaussian elimination without pivoting is numerically unstable.

6. *An Unstable Algorithm for Eigenvalue Computations:* Finding the eigenvalues of a matrix by finding the roots of its characteristic polynomial is a numerically unstable process because the roots of the characteristic polynomial may be ill-conditioned when the eigenvalues of the corresponding matrix are well-conditioned. Transforming a matrix to companion form often requires an ill-conditioned similarity transformation, so even calculating the coefficients of the characteristic polynomial may be an unstable process. A well-known example is the diagonal matrix $A = \text{diag}(1, 2, 3, \dots, 20)$. The Wielandt-Hoffman theorem [GV96] shows that perturbing A to a nearby matrix $A + E$ perturbs the eigenvalues by no more than $\|E\|_F$. However, the characteristic polynomial is the infamous Wilkinson polynomial discussed in Example 5 of Section 37.4, which has highly ill-conditioned roots.

Author Note

The contribution of Ralph Byers' material is partially supported by the National Sciences Foundation Award 0098150.

References

- [Bha96] R. Bhatia, *Matrix Analysis*, Springer, New York, 1996.
- [Bun87] J. Bunch, The weak and strong stability of algorithms in numerical linear algebra, *Linear Algebra and Its Applications*, 88, 49–66, 1987.
- [Dat95] B.N. Datta, *Numerical Linear Algebra and Applications*, Brooks/Cole Publishing Company, Pacific Grove, CA, 1995. (Section edition to be published in 2006.)
- [GV96] G.H. Golub and C.F. Van Loan, *Matrix Computations*, 3rd ed., Johns Hopkins University Press, Baltimore, MD, 1996.
- [Hig96] N.J. Higham, *Accuracy and Stability of Numerical Algorithms*, SIAM, Philadelphia, 1996.
- [HJ85] R.A. Horn and C.R. Johnson, *Matrix Analysis*, Cambridge University Press, New York, 1985.
- [IEEE754] IEEE 754-1985. "IEEE Standard for Binary Floating-Point Arithmetic" (ANSI/IEEE Std 754-1985), The Institute of Electrical and Electronics Engineers, Inc., New York, 1985. Reprinted in *SIGPLAN Notices*, 22(2): 9–25, 1987.
- [IEEE754r] (Draft) "Standard for Floating Point Arithmetic P754/D0.15.3—2005," October 20. The Institute of Electrical and Electronics Engineers, Inc., New York, 2005.
- [IEEE854] IEEE 854-1987. "IEEE Standard for Radix-Independent Floating-Point Arithmetic." The Institute of Electrical and Electronics Engineers, Inc., New York, 1987.
- [Kat66] T. Kato, *Perturbation Theory for Linear Operators*, Springer-Verlag, New York, 1966. (A corrected second edition appears in 1980, which was republished in the *Classics in Mathematics* series in 1995.)
- [Ove01] M. Overton, *Numerical Computing with IEEE Floating Point Arithmetic*, Society for Industrial and Applied Mathematics, Philadelphia, PA, 2001.
- [Ric66] J. Rice, A theory of condition, *SIAM J. Num. Anal.*, 3, 287–310, 1966.
- [Ste98] G.W. Stewart, *Matrix Algorithms, Vol. 1, Basic Decompositions*, SIAM, Philadelphia, 1998.
- [SS90] G.W. Stewart and J.G. Sun, *Matrix Perturbation Theory*, Academic Press, New York, 1990.
- [TB97] L.N. Trefethan and D. Bau, *Numerical Linear Algebra*, SIAM, Philadelphia, 1997.
- [Wil65] J.H. Wilkinson, *The Algebraic Eigenvalue Problem*, Clarendon Press, Oxford, U.K., 1965.
- [Wil64] J.H. Wilkinson, *Rounding Errors in Algebraic Processes*, Prentice-Hall, Inc., Upper Saddle River, NJ, 1963. Reprinted by Dover Publications, Mineola, NY, 1994.

38

Matrix Factorizations and Direct Solution of Linear Systems

Christopher Beattie

*Virginia Polytechnic Institute and State
University*

38.1	Perturbations of Linear Systems	38-2
38.2	Triangular Linear Systems	38-5
38.3	Gauss Elimination and LU Decomposition	38-7
38.4	Orthogonalization and QR Decomposition	38-13
38.5	Symmetric Factorizations	38-15
	References	38-17

The need to solve systems of linear equations arises often within diverse disciplines of science, engineering, and finance. The expression “direct solution of linear systems” refers generally to computational strategies that can produce solutions to linear systems after a predetermined number of arithmetic operations that depends only on the structure and dimension of the coefficient matrix. The evolution of computers has and continues to influence the development of these strategies as well as fostering particular styles of perturbation analysis suited to illuminating their behavior. Some general themes have become dominant, as a result; others have been pushed aside. For example, Cramer’s Rule may be properly thought of as a direct solution strategy for solving linear systems; however it requires a much larger number of arithmetic operations than Gauss elimination and is generally much more susceptible to the deleterious effects of rounding. Most current approaches for the direct solution of a linear system, $A\mathbf{x} = \mathbf{b}$, are patterned after Gauss elimination and favor systematically decoupling the system of equations. Zeros are introduced systematically into the coefficient matrix, transforming it into triangular form; the resulting triangular system is easily solved. The entire process can be viewed in this way:

1. Find invertible matrices $\{S_i\}_{i=1}^p$ such that $S_p \dots S_2 S_1 A = U$ is triangular; then
2. Calculate a modified right-hand side $\mathbf{y} = S_p \dots S_2 S_1 \mathbf{b}$; and then
3. Determine the solution set to the triangular system $U\mathbf{x} = \mathbf{y}$.

The matrices $S_1, S_2, \dots, S_p$ are typically either row permutations of lower triangular matrices (Gauss transformations) or unitary matrices and so have readily available inverses. Evidently A can be written as $A = NU$, where $N = (S_p \dots S_2 S_1)^{-1}$. A solution framework may be built around the availability of decompositions such as this:

1. Find a decomposition $A = NU$ such that U is triangular and $N\mathbf{y} = \mathbf{b}$ is easily solved;
2. Solve $N\mathbf{y} = \mathbf{b}$; then
3. Determine the solution set to triangular system $U\mathbf{x} = \mathbf{y}$.

38.1 Perturbations of Linear Systems

In the computational environment afforded by current computers, the finite representation of real numbers creates a small but persistent source of errors that may on occasion severely degrade the overall accuracy of a calculation. This effect is of fundamental concern in assessing strategies for solving linear systems.

Rounding errors can be introduced into the solution process for linear systems often before any calculations are performed — as soon as data are stored within the computer and represented within the internal floating point number system of the computer. Further errors that may be introduced in the course of computation often may be viewed in aggregate as an effective additional contribution to the initial representation error. Inevitably then the linear system for which a solution is computed deviates slightly from the “true” linear system and it becomes of critical interest to determine whether such deviations will have a significant effect on the accuracy of the final computed result.

Definitions:

Let $A \in \mathbb{C}^{n \times n}$ be a nonsingular matrix, $\mathbf{b} \in \mathbb{C}^n$, and then denote by $\hat{\mathbf{x}} = A^{-1}\mathbf{b}$ the unique solution of the linear system $A\mathbf{x} = \mathbf{b}$.

Given **data perturbations** $\delta A \in \mathbb{C}^{n \times n}$ and $\delta \mathbf{b} \in \mathbb{C}^n$ to A and $\mathbf{b}$, respectively, the **solution perturbation**, $\delta \mathbf{x} \in \mathbb{C}^n$ satisfies the associated **perturbed linear system** $(A + \delta A)(\hat{\mathbf{x}} + \delta \mathbf{x}) = \mathbf{b} + \delta \mathbf{b}$ (presuming then that the perturbed system is consistent).

For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$, the **residual vector** associated with the linear system $A\mathbf{x} = \mathbf{b}$ is defined as $\mathbf{r}(\tilde{\mathbf{x}}) = \mathbf{b} - A\tilde{\mathbf{x}}$.

For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$, the associated (norm-wise) **relative backward error of the linear system** $A\mathbf{x} = \mathbf{b}$ (with respect to the the p -norm) is

$$\eta_p(A, \mathbf{b}; \tilde{\mathbf{x}}) = \min \left\{ \varepsilon \left| \begin{array}{l} \text{there exist } \delta A, \delta \mathbf{b} \text{ such that} \\ (A + \delta A)\tilde{\mathbf{x}} = \mathbf{b} + \delta \mathbf{b} \text{ with} \end{array} \right. \begin{array}{l} \|\delta A\|_p \leq \varepsilon \|A\|_p \\ \|\delta \mathbf{b}\|_p \leq \varepsilon \|\mathbf{b}\|_p \end{array} \right\}$$

for $1 \leq p \leq \infty$.

For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$, the associated **component-wise relative backward error of the linear system** $A\mathbf{x} = \mathbf{b}$ is

$$\omega(A, \mathbf{b}; \tilde{\mathbf{x}}) = \min \left\{ \varepsilon \left| \begin{array}{l} \text{there exist } \delta A, \delta \mathbf{b} \text{ such that} \\ (A + \delta A)\tilde{\mathbf{x}} = \mathbf{b} + \delta \mathbf{b} \text{ with} \end{array} \right. \begin{array}{l} |\delta A| \leq \varepsilon |A| \\ |\delta \mathbf{b}| \leq \varepsilon |\mathbf{b}| \end{array} \right\},$$

where the absolute values and inequalities applied to vectors and matrices are interpreted component-wise: $|B| \leq |A|$ means $|b_{ij}| \leq |a_{ij}|$ for all index pairs i, j .

The (norm-wise) **condition number of the linear system** $A\mathbf{x} = \mathbf{b}$ (relative to the the p -norm) is

$$\kappa_p(A, \hat{\mathbf{x}}) = \|A^{-1}\|_p \frac{\|\mathbf{b}\|_p}{\|\hat{\mathbf{x}}\|_p}$$

for $1 \leq p \leq \infty$.

The **matrix condition number** relative to the the p -norm of A is

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p$$

for $1 \leq p \leq \infty$.

The **Skeel condition number of the linear system** $A\mathbf{x} = \mathbf{b}$ is

$$\text{cond}(A, \hat{\mathbf{x}}) = \frac{\| |A^{-1}| |A| |\hat{\mathbf{x}}| \|_\infty}{\|\hat{\mathbf{x}}\|_\infty}.$$

The **Skeel matrix condition number** is $\text{cond}(A) = \| |A^{-1}| |A| \|_\infty$.

Facts: [Hig96],[StS90]

1. For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$, $\tilde{\mathbf{x}}$ is the exact solution to any one of the family of perturbed linear systems

$$(A + \delta A_\theta) \tilde{\mathbf{x}} = \mathbf{b} + \delta \mathbf{b}_\theta,$$

where $\theta \in \mathbb{C}$, $\delta \mathbf{b}_\theta = (\theta - 1) \mathbf{r}(\tilde{\mathbf{x}})$, $\delta A_\theta = \theta \mathbf{r}(\tilde{\mathbf{x}}) \tilde{\mathbf{y}}^*$, and $\tilde{\mathbf{y}} \in \mathbb{C}^n$ is any vector such that $\tilde{\mathbf{y}}^* \tilde{\mathbf{x}} = 1$. In particular, for $\theta = 0$, $\delta A = 0$ and $\delta \mathbf{b} = -\mathbf{r}(\tilde{\mathbf{x}})$; for $\theta = 1$, $\delta A = \mathbf{r}(\tilde{\mathbf{x}}) \tilde{\mathbf{y}}^*$ and $\delta \mathbf{b} = 0$.

2. (Rigal–Gaches Theorem) For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$,

$$\eta_p(A, \mathbf{b}; \tilde{\mathbf{x}}) = \frac{\|\mathbf{r}(\tilde{\mathbf{x}})\|_p}{\|A\|_p \|\tilde{\mathbf{x}}\|_p + \|\mathbf{b}\|_p}.$$

If $\tilde{\mathbf{y}}$ is the dual vector to $\tilde{\mathbf{x}}$ with respect to the p -norm ($\tilde{\mathbf{y}}^* \tilde{\mathbf{x}} = \|\tilde{\mathbf{y}}\|_q \|\tilde{\mathbf{x}}\|_p = 1$ with $\frac{1}{p} + \frac{1}{q} = 1$), then $\tilde{\mathbf{x}}$ is an exact solution to the perturbed linear system $(A + \delta A_{\tilde{\theta}}) \tilde{\mathbf{x}} = \mathbf{b} + \delta \mathbf{b}_{\tilde{\theta}}$ with data perturbations as in (1) and $\tilde{\theta} = \frac{\|A\|_p \|\tilde{\mathbf{x}}\|_p}{\|A\|_p \|\tilde{\mathbf{x}}\|_p + \|\mathbf{b}\|_p}$, and as a result

$$\frac{\|\delta A_{\tilde{\theta}}\|_p}{\|A\|_p} = \frac{\|\delta \mathbf{b}_{\tilde{\theta}}\|_p}{\|\mathbf{b}\|_p} = \eta_p(A, \mathbf{b}; \tilde{\mathbf{x}}).$$

3. (Oettli–Prager Theorem) For any $\tilde{\mathbf{x}} \in \mathbb{C}^n$,

$$\omega(A, \mathbf{b}; \tilde{\mathbf{x}}) = \max_i \frac{|r_i|}{(|A| |\tilde{\mathbf{x}}| + |\mathbf{b}|)_i}.$$

If $D_1 = \text{diag} \left(\frac{r_i}{(|A| |\tilde{\mathbf{x}}| + |\mathbf{b}|)_i} \right)$ and $D_2 = \text{diag}(\text{sign}(\tilde{\mathbf{x}})_i)$, then $\tilde{\mathbf{x}}$ is an exact solution to the perturbed linear system $(A + \delta A) \tilde{\mathbf{x}} = \mathbf{b} + \delta \mathbf{b}$ with $\delta A = D_1 |A| D_2$ and $\delta \mathbf{b} = -D_1 |\mathbf{b}|$

$$|\delta A| \leq \omega(A, \mathbf{b}; \tilde{\mathbf{x}}) |A| \quad \text{and} \quad |\delta \mathbf{b}| \leq \omega(A, \mathbf{b}; \tilde{\mathbf{x}}) |\mathbf{b}|$$

and no smaller constant can be used in place of $\omega(A, \mathbf{b}; \tilde{\mathbf{x}})$.

4. The reciprocal of $\kappa_p(A)$ is the smallest norm-wise relative distance of A to a singular matrix, i.e.,

$$\frac{1}{\kappa_p(A)} = \min \left\{ \frac{\|\delta A\|_p}{\|A\|_p} \mid A + \delta A \text{ is singular} \right\}.$$

In particular, the perturbed coefficient matrix $A + \delta A$ is nonsingular if

$$\frac{\|\delta A\|_p}{\|A\|_p} < \frac{1}{\kappa_p(A)}.$$

5. $1 \leq \kappa_p(A, \hat{\mathbf{x}}) \leq \kappa_p(A)$ and $1 \leq \text{cond}(A, \hat{\mathbf{x}}) \leq \text{cond}(A) \leq \kappa_\infty(A)$.
6. $\text{cond}(A) = \min \{ \kappa_\infty(DA) \mid D \text{ diagonal} \}$.
7. If $\delta A = 0$, then

$$\frac{\|\delta \mathbf{x}\|_p}{\|\hat{\mathbf{x}}\|_p} \leq \kappa_p(A, \hat{\mathbf{x}}) \frac{\|\delta \mathbf{b}\|_p}{\|\mathbf{b}\|_p}.$$

8. If $\delta \mathbf{b} = 0$ and $A + \delta A$ is nonsingular, then

$$\frac{\|\delta \mathbf{x}\|_p}{\|\hat{\mathbf{x}} + \delta \mathbf{x}\|_p} \leq \kappa_p(A) \frac{\|\delta A\|_p}{\|A\|_p}.$$

9. If $\|\delta A\|_p \leq \epsilon \|A\|_p$, $\|\delta \mathbf{b}\|_p \leq \epsilon \|\mathbf{b}\|_p$, and $\epsilon < \frac{1}{\kappa_p(A)}$, then

$$\frac{\|\delta \mathbf{x}\|_p}{\|\hat{\mathbf{x}}\|_p} \leq \frac{2 \epsilon \kappa_p(A)}{1 - \epsilon \kappa_p(A)}.$$

10. If $|\delta A| \leq \epsilon |A|$, $|\delta \mathbf{b}| \leq \epsilon |\mathbf{b}|$, and $\epsilon < \frac{1}{\text{cond}(A)}$, then

$$\frac{\|\delta \mathbf{x}\|_\infty}{\|\hat{\mathbf{x}}\|_\infty} \leq \frac{2\epsilon \text{cond}(A, \hat{\mathbf{x}})}{1 - \epsilon \text{cond}(A)}.$$

Examples:

1. Let $A = \begin{bmatrix} 1000 & 999 \\ 999 & 998 \end{bmatrix}$ so $A^{-1} = \begin{bmatrix} -998 & 999 \\ 999 & -1000 \end{bmatrix}$. Then $\|A\|_1 = \|A^{-1}\|_1 = 1999$ so that $\kappa_1(A) \approx 3.996 \times 10^6$. Consider

$$\mathbf{b} = \begin{bmatrix} 1999 \\ 1997 \end{bmatrix} \text{ associated with a solution } \hat{\mathbf{x}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

A perturbation of right-hand side $\delta \mathbf{b} = \begin{bmatrix} -0.01 \\ 0.01 \end{bmatrix}$ constitutes a relative change in the right-hand side of $\frac{\|\delta \mathbf{b}\|_1}{\|\mathbf{b}\|_1} \approx 5.005 \times 10^{-6}$ yet it produces a perturbed solution $\hat{\mathbf{x}} + \delta \mathbf{x} = \begin{bmatrix} 20.97 \\ -18.99 \end{bmatrix}$ constituting a relative change $\frac{\|\delta \mathbf{x}\|_1}{\|\hat{\mathbf{x}}\|_1} = 19.98 \leq 20 = \kappa_1(A) \frac{\|\delta \mathbf{b}\|_1}{\|\mathbf{b}\|_1}$. The bound determined by the condition number is very nearly achieved. Note that the same perturbed solution $\hat{\mathbf{x}} + \delta \mathbf{x}$ could be produced by a change in the coefficient matrix

$$\delta A = \tilde{\mathbf{r}}\tilde{\mathbf{y}}^* = - \begin{bmatrix} -0.01 \\ 0.01 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 39.96 & -39.96 \end{bmatrix} = (1/3996) \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

constituting a relative change $\frac{\|\delta A\|_1}{\|A\|_1} \approx 2.5 \times 10^{-7}$. Then $(A + \delta A)(\hat{\mathbf{x}} + \delta \mathbf{x}) = \mathbf{b}$.

2. Let $n = 100$ and A be tridiagonal with diagonal entries equal to -2 and all superdiagonal and subdiagonal entries equal to 1 (associated with a centered difference approximation to the second derivative). Let $\mathbf{b}$ be a vector with a quadratic variation in entries

$$b_k = (k-1)(100-k)/10,000.$$

Then

$$\kappa_2(A, \hat{\mathbf{x}}) \approx 1, \quad \text{but} \quad \kappa_2(A) \approx 4.1336 \times 10^3.$$

Since the elements of $\mathbf{b}$ do not have an exact binary representation, the linear system that is presented to any computational algorithm will be $A\mathbf{x} = \mathbf{b} + \delta \mathbf{b}$ with $\|\delta \mathbf{b}\|_2 \leq \epsilon \|\mathbf{b}\|_2$, where ϵ is the unit roundoff error. For example, if the linear system data is stored in IEEE single precision format, $\epsilon \approx 6 \times 10^{-8}$. The matrix condition number, $\kappa_2(A)$, would yield a bound of $(6 \times 10^{-8})(4.1336 \times 10^3) \approx 2.5 \times 10^{-4}$ anticipating the loss of more than 4 significant digits in solution components even if all computations were done on the stored data with no further error. However, the condition number of the linear system, $\kappa_2(A, \hat{\mathbf{x}})$, is substantially smaller and the predicted error for the system is roughly the same as the initial representation error $\approx 6 \times 10^{-8}$, indicating that the solution will be fairly insensitive to the consequences of rounding of the right-hand side data—assuming no further errors occur. But, in fact, this conclusion remains true even if further errors occur, if whatever computational algorithm that is used produces small backward error, as might be asserted if, say, a final residual satisfies $\|\mathbf{r}\|_2 \leq \mathcal{O}(\epsilon) \|\mathbf{b}\|_2$. This situation changes substantially if the right-hand side is changed to

$$b_k = (-1)^k(k-1)(100-k)/10,000,$$

which only introduces a sign variation in $\mathbf{b}$. In this case, $\kappa_2(A, \hat{\mathbf{x}}) \approx \kappa_2(A)$, and the components of the computed solution can be expected to lose about 4 significant digits purely on the basis of errors that are made in the initial representation. Additional errors made in the course of the computation can hardly be expected to improve this situation.

38.2 Triangular Linear Systems

Systems of linear equations for which the unknowns may be solved for one at a time in sequence may be reordered to produce linear systems with triangular coefficient matrices. Such systems can be solved both with remarkable accuracy and remarkable efficiency. Triangular systems are the archetype for easily solvable systems of linear equations; as such, they often constitute an intermediate goal for strategies of solving linear systems.

Definitions:

A linear system of equations $T\mathbf{x} = \mathbf{b}$ with $T \in \mathbb{C}^{n \times n}$ (representing n equations in n unknowns) is a **triangular system** if $T = [t_{ij}]$ is either an **upper triangular matrix** ($t_{ij} = 0$ for $i > j$) or a **lower triangular matrix** ($t_{ij} = 0$ for $i < j$).

Facts: [Hig96], [GV96]

1. [GV96, pp. 88–90]

Algorithm 1: Row-oriented forward-substitution for solving lower triangular system

Input: $L \in \mathbb{R}^{n \times n}$ with $\ell_{ij} = 0$ for $i < j$; $\mathbf{b} \in \mathbb{R}^n$

Output: solution vector $\mathbf{x} \in \mathbb{R}^n$ that satisfies $L\mathbf{x} = \mathbf{b}$

$x_1 \leftarrow b_1/\ell_{1,1}$

For $k = 2 : n$

$x_k \leftarrow (b_k - L_{k,1:k-1} \cdot x_{1:k-1})/\ell_{k,k}$

end

2. [GV96, pp. 88–90]

Algorithm 2: Column-oriented back-substitution for solving upper triangular system

Input: $U \in \mathbb{R}^{n \times n}$ with $u_{ij} = 0$ for $i > j$; $\mathbf{b} \in \mathbb{R}^n$

Output: solution vector $\mathbf{x} \in \mathbb{R}^n$ that satisfies $U\mathbf{x} = \mathbf{b}$

For $k = n$ down to 2 by steps of -1

$x_k \leftarrow b_k/u_{k,k}$

$b_{1:k-1} \leftarrow b_{1:k-1} - x_k U_{1:k-1,k}$

end

$x_1 \leftarrow b_1/u_{1,1}$

3. Algorithm 1 involves as a core calculation, dot products of portions of coefficient matrix rows with corresponding portions of the emerging solution vector. This can incur a performance penalty for large n from accumulation of dot products using a scalar recurrence. A “column-oriented” reformulation may have better performance for large n . Algorithm 2 is a “column-oriented” formulation for solving upper triangular systems.

4. The solution of triangular systems using either Algorithm 1 or 2 is *componentwise backward stable*. In particular the computed result, $\tilde{\mathbf{x}}$, produced either by Algorithm 1 or 2 in solving a triangular system, $T\mathbf{x} = \mathbf{b}$, will be the exact result of a perturbed system $(T + \delta T)\tilde{\mathbf{x}} = \mathbf{b}$, where $|\delta T| \leq \frac{n\epsilon}{1 - n\epsilon}|T|$ and ϵ is the unit roundoff error.
5. The error in the solution of a triangular system, $T\mathbf{x} = \mathbf{b}$, using either Algorithm 1 or 2 satisfies

$$\frac{\|\tilde{\mathbf{x}} - \hat{\mathbf{x}}\|_{\infty}}{\|\hat{\mathbf{x}}\|_{\infty}} \leq \frac{n\epsilon \text{cond}(T, \hat{\mathbf{x}})}{1 - n\epsilon (\text{cond}(T) + 1)}.$$

6. If $T = [t_{ij}]$ is an lower triangular matrix satisfying $|t_{ii}| \geq |t_{ij}|$ for $j \leq i$, the computed solution to the linear system $T\mathbf{x} = \mathbf{b}$ produced by either Algorithm 1 or the variant of Algorithm 2 for lower triangular systems satisfies

$$|\hat{x}_i - \tilde{x}_i| \leq \frac{2^i n\epsilon}{1 - n\epsilon} \max_{j \leq i} |\tilde{x}_j|,$$

where $\tilde{x}_i$ are the components of the computed solution, $\tilde{\mathbf{x}}$, and $\hat{x}_i$ are the components of the exact solution, $\hat{\mathbf{x}}$. Although this bound degrades exponentially with i , it shows that early solution components will be computed to high accuracy relative to those components already computed.

Examples:

1. Use Algorithm 2 to solve the triangular system

$$\begin{bmatrix} 1 & 2 & -3 \\ 0 & 2 & -6 \\ 0 & 0 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}.$$

$k = 3$ step: Solve for $x_3 = 1/3$. Update right-hand side:

$$\begin{bmatrix} 1 & 2 \\ 0 & 2 \\ 0 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - (1/3) \begin{bmatrix} -3 \\ -6 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix}.$$

$k = 2$ step: Solve for $x_2 = 3/2$. Update right-hand side:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} [x_1] = \begin{bmatrix} 2 \\ 3 \\ 0 \end{bmatrix} - (3/2) \begin{bmatrix} 2 \\ 2 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 0 \end{bmatrix}.$$

$k = 1$ step: Solve for $x_1 = -1$.

2. ([Hig96, p. 156]) For $\epsilon > 0$, consider $T = \begin{bmatrix} 1 & 0 & 0 \\ \epsilon & \epsilon & 0 \\ 0 & 1 & 1 \end{bmatrix}$. Then $T^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ -1 & \frac{1}{\epsilon} & 0 \\ 1 & -\frac{1}{\epsilon} & 1 \end{bmatrix}$, and so $\text{cond}(T) = 5$, even though

$$\kappa_{\infty}(T) = 2(2 + \frac{1}{\epsilon}) \approx \frac{2}{\epsilon} + \mathcal{O}(1).$$

Thus, linear systems having T as a coefficient matrix will be solved to high relative accuracy, independent of both right-hand side and size of ϵ , despite the poor conditioning of T (as measured by κ_{∞}) as ϵ becomes small. However, note that

$$\text{cond}(T^T) = 1 + \frac{2}{\epsilon} \quad \text{and} \quad \kappa_{\infty}(T^T) = (1 + \epsilon)\frac{2}{\epsilon} \approx \frac{2}{\epsilon} + \mathcal{O}(1).$$

So, linear systems having T^T as a coefficient matrix may have solutions that are sensitive to perturbations and indeed, $\text{cond}(T^T, \mathbf{x}) \approx \text{cond}(T^T)$ for any right-hand side $\mathbf{b}$ with $b_3 \neq 0$ yielding solutions that are sensitive to perturbations for small ϵ .

38.3 Gauss Elimination and LU Decomposition

Gauss elimination is an elementary approach to solving systems of linear equations, yet it still constitutes the core of the most sophisticated of solution strategies. In the k^{th} step, a transformation matrix, M_k , (a “Gauss transformation”) is designed so as to introduce zeros into A — typically into a portion of the k^{th} column — without harming zeros that have been introduced in earlier steps. Typically, successive applications of Gauss transformations are interleaved with row interchanges. Remarkably, this reduction process can be viewed as producing a decomposition of the coefficient matrix $A = NU$, where U is a triangular matrix and N is a row permutation of a lower triangular matrix.

Definitions:

For each index k , a **Gauss vector** is a vector in $\mathbb{C}^n$ with the leading k entries equal to zero: $\ell_k = [0, \dots, 0, \underbrace{\ell_{k+1}, \dots, \ell_n}_k]^T$. The entries $\ell_{k+1}, \dots, \ell_n$ are **Gauss multipliers** and the associated matrix

$$M_k = I - \ell_k \mathbf{e}_k^T$$

is called a **Gauss transformation**.

For the pair of indices (i, j) , with $i \leq j$ the associated **permutation matrix**, $\Pi_{i,j}$ is an $n \times n$ identity matrix with the i^{th} row and j^{th} row interchanged. Note that $\Pi_{i,i}$ is the identity matrix.

A matrix $U \in \mathbb{C}^{m \times n}$ is in **row-echelon form** if (1) the first nonzero entry of each row has a strictly smaller column index than all nonzero entries having a strictly larger row index and (2) zero rows occur at the bottom. The first nonzero entry in each row of U is called a **pivot**. Thus, the determining feature of row echelon form is that pivots occur to the left of all nonzero entries in lower rows.

A matrix $A \in \mathbb{C}^{m \times n}$ has an **LU decomposition** if there exists a unit lower triangular matrix $L \in \mathbb{C}^{m \times m}$ ($L_{i,j} = 0$ for $i < j$ and $L_{i,i} = 1$ for all i) and an upper triangular matrix $U \in \mathbb{C}^{m \times n}$ ($U_{i,j} = 0$ for $i > j$) such that $A = LU$.

Facts: [GV96]

1. Let $\mathbf{a} \in \mathbb{C}^n$ be a vector with a nonzero component in the r^{th} entry, $a_r \neq 0$. Define the Gauss vector, $\ell_r = [0, \dots, 0, \underbrace{a_{r+1}, \dots, a_n}_r, \frac{a_{r+1}}{a_r}, \dots, \frac{a_n}{a_r}]^T$. The associated Gauss transformation $M_r = I - \ell_r \mathbf{e}_r^T$ introduces zeros into the last $n - r$ entries of $\mathbf{a}$:

$$M_r \mathbf{a} = [a_1, \dots, a_r, 0, \dots, 0]^T.$$

2. If $A \in \mathbb{C}^{m \times n}$ with $\text{rank}(A) = \rho \geq 1$ has ρ leading principal submatrices nonsingular, $A_{1:r, 1:r}$, $r = 1, \dots, \rho$, then there exist Gauss transformations $M_1, M_2, \dots, M_\rho$ so that

$$M_\rho M_{\rho-1} \cdots M_1 A = U$$

with U upper triangular. Each Gauss transformation M_r introduces zeros into the r^{th} column.

3. Gauss transformations are unit lower triangular matrices. They are invertible, and for the Gauss transformation, $M_r = I - \ell_r \mathbf{e}_r^T$,

$$M_r^{-1} = I + \ell_r \mathbf{e}_r^T.$$

4. If Gauss vectors $\ell_1, \ell_2, \dots, \ell_{n-1}$ are given with

$$\ell_1 = \begin{Bmatrix} 0 \\ \ell_{21} \\ \ell_{31} \\ \vdots \\ \ell_{n1} \end{Bmatrix}, \quad \ell_2 = \begin{Bmatrix} 0 \\ 0 \\ \ell_{32} \\ \vdots \\ \ell_{n2} \end{Bmatrix}, \quad \dots, \quad \ell_{n-1} = \begin{Bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ \ell_{n,n-1} \end{Bmatrix},$$

then the product of Gauss transformations $M_{n-1}M_{n-2} \cdots M_2M_1$ is invertible and has an explicit inverse

$$(M_{n-1}M_{n-2} \cdots M_2M_1)^{-1} = I + \sum_{k=1}^{n-1} \ell_k \mathbf{e}_k^T = \begin{bmatrix} 1 & 0 & \cdots & 0 & 0 \\ \ell_{21} & 1 & & & 0 \\ \ell_{31} & \ell_{32} & \ddots & & 0 \\ \vdots & & & 1 & 0 \\ \ell_{n1} & \ell_{n2} & \cdots & \ell_{n,n-1} & 1 \end{bmatrix}.$$

5. If $A \in \mathbb{C}^{m \times n}$ with $\text{rank}(A) = \rho$ has ρ leading principal submatrices nonsingular, $A_{1:r,1:r}$, $r = 1, \dots, \rho$, then A has an LU decomposition: $A = LU$, with L unit lower triangular and U upper triangular. The (i, j) entry of L , $L_{i,j}$ with $i > j$ is the Gauss multiplier used to introduce a zero into the corresponding (i, j) entry of A . If, additionally, $\rho = m$, then the LU decomposition is unique.
6. Let $\mathbf{a}$ be an arbitrary vector in $\mathbb{C}^n$. For any index r , there is an index $\mu \geq r$, a permutation matrix $\Pi_{r,\mu}$, and a Gauss transformation M_r so that

$$M_r \Pi_{r,\mu} \mathbf{a} = [a_1, \dots, a_{r-1}, a_\mu, \underbrace{0, \dots, 0}_{n-r}]^T.$$

The index μ is chosen so that $a_\mu \neq 0$ out of the set $\{a_r, a_{r+1}, \dots, a_n\}$. If $a_r \neq 0$, then $\mu = k$ and $\Pi_{r,\mu} = I$ is a possible choice; if each element is zero, $a_r = a_{r+1} = \cdots = a_n = 0$, then $\mu = k$, $\Pi_{r,\mu} = I$, and $M_r = I$ is a possible choice.

7. For every matrix $A \in \mathbb{C}^{m \times n}$ with $\text{rank}(A) = \rho$, there exists a sequence of ρ indices $\mu_1, \mu_2, \dots, \mu_\rho$ with $i \leq \mu_i \leq m$ for $i = 1, \dots, \rho$ and Gauss transformations $M_1, M_2, \dots, M_\rho$ so that

$$M_\rho \Pi_{\rho,\mu_\rho} M_{\rho-1} \Pi_{\rho-1,\mu_{\rho-1}} \cdots M_1 \Pi_{1,\mu_1} A = U$$

with U upper triangular and in row echelon form. Each pair of transformations $M_r \Pi_{r,\mu_r}$ introduces zeros below the r^{th} pivot.

8. For $r < i < j$, $\Pi_{i,j} M_r = \tilde{M}_r \Pi_{i,j}$, where $\tilde{M}_r = I - \tilde{\ell}_r \mathbf{e}_r^T$ and $\tilde{\ell}_r = \Pi_{i,j} \ell_r$ (i.e., the i and j entries of ℓ_r are interchanged to form $\tilde{\ell}_r$).
9. For every matrix $A \in \mathbb{C}^{m \times n}$ with $\text{rank}(A) = \rho$, there is a row permutation of A that has an LU decomposition: $PA = LU$, with a permutation matrix P , unit lower triangular matrix L , and an upper triangular matrix U that is in row echelon form. P can be chosen as $P = \Pi_{\rho,\mu_\rho} \Pi_{\rho-1,\mu_{\rho-1}} \cdots \Pi_{1,\mu_1}$ from (7), though in general there can be many other possibilities as well.
10. Reduction of A with Gauss transformations (or equivalently, calculation of an LU factorization) must generally incorporate row interchanges. As a practical matter, these row interchanges commonly are chosen so as to bring the largest magnitude entry within the column being reduced up into the pivot location. This strategy is called “partial pivoting.” In particular, if zeros are to be introduced into the k^{th} column below the r^{th} row (with $r \leq k$), then one seeks an index μ_r such that $r \leq \mu_r \leq m$ and $|A_{\mu_r,k}| = \max_{r \leq i \leq m} |A_{i,k}|$. When $\mu_1, \mu_2, \dots, \mu_\rho$ in (7) are chosen in this way, the reduction process is called “Gaussian Elimination with Partial Pivoting” (GEPP) or, within the context of factorization, the permuted LU factorization (PLU).

11. [GV96, p. 115]

Algorithm 1: GEPP/PLU decomposition of a rectangular matrix (outer product)

Input: $A \in \mathbb{R}^{m \times n}$
Output: $L \in \mathbb{R}^{m \times m}$ (unit lower triangular matrix)
 $U \in \mathbb{R}^{m \times n}$ (upper triangular matrix - row echelon form)
 $P \in \mathbb{R}^{m \times m}$ (permutation matrix) so that $PA = LU$
(P is represented with an index vector $\mathbf{p}$ such that $\mathbf{y} = P\mathbf{z} \Leftrightarrow y_j = z_{p_j}$)
 $L \leftarrow I_m$; $U \leftarrow 0 \in \mathbb{R}^{m \times n}$;
 $\mathbf{p} = [1, 2, 3, \dots, m]$
 $r \leftarrow 1$;
For $k = 1$ to n
Find μ such that $r \leq \mu \leq m$ and $|A_{\mu,k}| = \max_{r \leq i \leq m} |A_{i,k}|$
If $A_{\mu,k} \neq 0$, then
Exchange $A_{\mu,k:n} \leftrightarrow A_{r,k:n}$, $L_{\mu,1:r-1} \leftrightarrow L_{r,1:r-1}$, and $p_\mu \leftrightarrow p_r$
 $L_{r+1:m,r} \leftarrow A_{r+1:m,r} / A_{r,k}$
 $U_{r,k:n} \leftarrow A_{r,k:n}$
For $i = r + 1$ to m
For $j = k + 1$ to n
 $A_{i,j} \leftarrow A_{i,j} - L_{i,r} U_{r,j}$
 $r \leftarrow r + 1$

12. [GV96, p. 115]

Algorithm 2: GEPP/PLU decomposition of a rectangular matrix (gaxpy)

Input: $A \in \mathbb{R}^{m \times n}$
Output: $L \in \mathbb{R}^{m \times m}$ (unit lower triangular matrix),
 $U \in \mathbb{R}^{m \times n}$ (upper triangular matrix - row echelon form), and
 $P \in \mathbb{R}^{m \times m}$ (permutation matrix) so that $PA = LU$
(P is represented with an index vector π that records row interchanges
 $\pi_r = \mu$ means row r and row $\mu > r$ were interchanged)
 $L \leftarrow I_m \in \mathbb{R}^{m \times m}$; $U \leftarrow 0 \in \mathbb{R}^{m \times n}$; and $r \leftarrow 1$;
For $j = 1$ to n
 $\mathbf{v} \leftarrow A_{1:m,j}$
If $r > 1$, then
for $i = 1$ to $r - 1$,
Exchange $v_i \leftrightarrow v_{\pi_i}$
Solve the triangular system, $L_{1:r-1,1:r-1} \cdot \mathbf{z} = \mathbf{v}_{1:r-1}$;
 $U_{1:r-1,j} \leftarrow \mathbf{z}$;
Update $\mathbf{v}_{r:m} \leftarrow \mathbf{v}_{r:m} - L_{r:m,1:r-1} \cdot \mathbf{z}$;
Find μ such that $|v_\mu| = \max_{r \leq i \leq m} |v_i|$
If $v_\mu \neq 0$, then
 $\pi_r \leftarrow \mu$
Exchange $v_\mu \leftrightarrow v_r$
For $i = 1$ to $r - 1$,
Exchange $L_{\mu,i} \leftrightarrow L_{r,i}$
 $L_{r+1:m,r} \leftarrow \mathbf{v}_{r+1:m} / v_r$
 $U_{r,j} \leftarrow v_r$
 $r \leftarrow r + 1$

13. The condition for skipping reduction steps ($A_{\mu,k} \neq 0$ in Algorithm 1 and $v_\mu \neq 0$ in Algorithm 2) indicates deficiency of column rank and the potential for an infinite number of solutions. These

conditions are sensitive to rounding errors that may occur in the calculation of those columns and as such, GEPP/PLU is applied for the most part in full column rank settings ($\text{rank}(A) = n$), guaranteeing that no zero pivots are encountered and that no reduction steps are skipped.

14. Both Algorithms 1 and 2 require approximately $\frac{2}{3}\rho^3 + \rho m(n - \rho) + \rho n(m - \rho)$ arithmetic operations. Algorithm 1 involves as a core calculation the updating of a submatrix having ever diminishing size. For large matrix dimension, the contents of this submatrix, $A_{r+1:m, k+1:n}$, may be widely scattered through computer memory and a performance penalty can occur in gathering the data for computation (which can be costly relative to the number of arithmetic operations that must be performed). Algorithm 2 is a reorganization that avoids excess data motion by delaying updates to columns until the step within which they have zeros introduced. This forces modifications to the matrix entries to be made just one column at a time and the necessary data motion can be more efficient.
15. Other strategies for avoiding the adverse effects of small pivots exist. Some are more aggressive than partial pivoting in producing the largest possible pivot, others are more restrained.

“Complete pivoting” uses both row and column permutations to bring in the largest possible pivot: If zeros are to be introduced into the k^{th} column in row entries $r + 1$ to m , then one seeks indices μ and ν such that $r \leq \mu \leq m$ and $k < \nu \leq n$ such that $|A_{\mu, \nu}| = \max_{\substack{r \leq i \leq m \\ k < j \leq n}} |A_{i, j}|$. Gauss elimination with complete pivoting produces a unit lower triangular matrix $L \in \mathbb{R}^{m \times m}$, an upper triangular matrix $U \in \mathbb{R}^{m \times n}$, and two permutation matrices, P and Q , so that $PAQ = LU$.

“Threshold pivoting” identifies pivot candidates in each step that achieve a significant (predetermined) fraction of the magnitude of the pivot that would have been used in that step for partial pivoting: Consider all $\hat{\mu}$ such that $r \leq \hat{\mu} \leq m$ and $|A_{\hat{\mu}, k}| \geq \tau \cdot \max_{r \leq i \leq m} |A_{i, k}|$, where $\tau \in (0, 1)$ is a given threshold. This allows pivots to be chosen on the basis of other criteria such as influence on sparsity while still providing some protection from instability. τ can often be chosen quite small ($\tau = 0.1$ or $\tau = 0.025$ are typical values).

16. If $\hat{P} \in \mathbb{R}^{m \times m}$, $\hat{L} \in \mathbb{R}^{m \times m}$, and $\hat{U} \in \mathbb{R}^{m \times n}$ are the computed permutation matrix and LU factors from either Algorithm 1 or 2 on $A \in \mathbb{R}^{m \times n}$, then

$$\hat{L}\hat{U} = \hat{P}(A + \delta A) \quad \text{with} \quad |\delta A| \leq \frac{2n\epsilon}{1-n\epsilon} |\hat{L}||\hat{U}|$$

and for the particular case that $m = n$ and A is nonsingular, if an approximate solution, $\hat{\mathbf{x}}$, to $A\mathbf{x} = \mathbf{b}$ is computed by solving the two triangular linear systems, $\hat{L}\mathbf{y} = \hat{P}\mathbf{b}$ and $\hat{U}\hat{\mathbf{x}} = \mathbf{y}$, then $\hat{\mathbf{x}}$ is the exact solution to a perturbed linear system:

$$(A + \delta A)\hat{\mathbf{x}} = \mathbf{b} \quad \text{with} \quad |\delta A| \leq \frac{2n\epsilon}{1-n\epsilon} \hat{P}^T |\hat{L}||\hat{U}|.$$

Furthermore, $|L_{i, j}| \leq 1$ and $|U_{i, j}| \leq 2^{i-1} \max_{k \leq i} |A_{k, j}|$, so

$$\|\delta A\|_{\infty} \leq \frac{2^n n^2 \epsilon}{1 - n\epsilon} \|A\|_{\infty}.$$

Examples:

1. Using Algorithm 1, find a permuted LU factorization of

$$A = \begin{bmatrix} 1 & 1 & 2 & 2 \\ 2 & 2 & 4 & 6 \\ -1 & -1 & -1 & 1 \\ 1 & 1 & 3 & 1 \end{bmatrix}.$$

Setup:	$\mathbf{p} = [1\ 2\ 3\ 4], r \leftarrow 1$
$k = 1$ step:	$\mu \leftarrow 2, \mathbf{p} = [2\ 1\ 3\ 4]$
	Permuted A: $\begin{bmatrix} 2 & 2 & 4 & 6 \\ 1 & 1 & 2 & 3 \\ -1 & -1 & -2 & 1 \\ 1 & 1 & 3 & 1 \end{bmatrix}$
LU snapshot:	$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & 0 \\ -\frac{1}{2} & 0 & 1 & 0 \\ \frac{1}{2} & 0 & 0 & 1 \end{bmatrix}$ and $U = \begin{bmatrix} 2 & 2 & 4 & 6 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$.
	Updated $A_{2:4,2:4}$: $\begin{bmatrix} 0 & 0 & -1 \\ 0 & 1 & 4 \\ 0 & 1 & -2 \end{bmatrix}$
	$r \leftarrow 2$
$k = 2$ step:	$\mu \leftarrow 2, A_{2,2} = \max_{2 \leq i \leq 4} A_{i,2} = 0$
$k = 3$ step:	$\mu \leftarrow 3, \mathbf{p} = [2\ 3\ 1\ 4], A_{3,3} = \max_{2 \leq i \leq 4} A_{i,3} = 1$
	Permuted $A_{2:4,3:4}$: $\begin{bmatrix} 1 & 4 \\ 0 & -1 \\ 1 & -2 \end{bmatrix}$
LU snapshot:	$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 1 & 0 \\ \frac{1}{2} & 1 & 0 & 1 \end{bmatrix}$ and $U = \begin{bmatrix} 2 & 2 & 4 & 6 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$.
	Updated $A_{3:4,4}$: $\begin{bmatrix} -1 \\ -6 \end{bmatrix}$
	$r \leftarrow 3$
$k = 4$ step:	$\mu \leftarrow 4, \mathbf{p} = [2\ 3\ 4\ 1], A_{4,4} = \max_{3 \leq i \leq 4} A_{i,4} = 6$
	Permuted $A_{2:4,3:4}$: $\begin{bmatrix} -6 \\ -1 \end{bmatrix}$
LU snapshot:	$L = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 & 0 \\ \frac{1}{2} & 1 & 1 & 0 \\ \frac{1}{2} & 0 & \frac{1}{6} & 1 \end{bmatrix}$ and $U = \begin{bmatrix} 2 & 2 & 4 & 6 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & -6 \\ 0 & 0 & 0 & 0 \end{bmatrix}$.

The permutation matrix associated with $\mathbf{p} = [2\ 3\ 4\ 1]$ is

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

and

$$PA = \begin{bmatrix} 2 & 2 & 4 & 6 \\ -1 & -1 & -1 & 1 \\ 1 & 1 & 3 & 1 \\ 1 & 1 & 2 & 2 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{1}{2} & 1 & 0 & 0 \\ \frac{1}{2} & 1 & 1 & 0 \\ \frac{1}{2} & 0 & \frac{1}{6} & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 & 4 & 6 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & -6 \\ 0 & 0 & 0 & 0 \end{bmatrix} = L \cdot U.$$

2. Using Algorithm 2, solve the system of linear equations

$$\begin{bmatrix} 1 & 3 & 1 \\ 2 & 2 & -1 \\ 2 & -1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ -3 \\ 3 \end{bmatrix}.$$

Phase 1: Find permuted LU decomposition.

$r \leftarrow 1$

$$j = 1 \text{ step: } \mathbf{v} \leftarrow \begin{bmatrix} 1 \\ 2 \\ 2 \end{bmatrix} . \pi_1 \leftarrow \mu = 2. \text{ Permuted } \mathbf{v}: \begin{bmatrix} 2 \\ 1 \\ 2 \end{bmatrix}$$

$$LU \text{ snapshot: } \pi = [2] \quad L = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} .$$

$r \leftarrow 2$

$$j = 2 \text{ step: } \mathbf{v} \leftarrow \begin{bmatrix} 3 \\ 2 \\ -1 \end{bmatrix} . \text{ Permuted } \mathbf{v}: \begin{bmatrix} 2 \\ 3 \\ -1 \end{bmatrix} .$$

$$\text{Solve } 1 \cdot \mathbf{z} = 2. U_{1,2} \leftarrow \mathbf{z} = [2].$$

$$\begin{bmatrix} \mathbf{v}_2 \\ \mathbf{v}_3 \end{bmatrix} \leftarrow \begin{bmatrix} 2 \\ -3 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix} - \begin{bmatrix} \frac{1}{2} \\ 1 \end{bmatrix} [2]$$

$$\pi_2 \leftarrow \mu = 3. L_{3,2} \leftarrow -\frac{2}{3} \text{ and } U_{2,2} \leftarrow -3$$

$$LU \text{ snapshot: } \pi = [2, 3] \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & 2 & 0 \\ 0 & -3 & 0 \\ 0 & 0 & 0 \end{bmatrix} .$$

$r \leftarrow 3$

$$j = 3 \text{ step: } \mathbf{v} \leftarrow \begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix} \text{ Permuted } \mathbf{v}: \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} . \text{ Solve } \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \cdot \mathbf{z} = \begin{bmatrix} -1 \\ 0 \end{bmatrix} ,$$

$$\begin{bmatrix} U_{1,3} \\ U_{2,3} \end{bmatrix} \leftarrow \mathbf{z} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} . v_3 \leftarrow 2\frac{1}{6} = 1 - [\frac{1}{2}, -\frac{2}{3}] \cdot \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

$$\pi_3 \rightarrow 3 \text{ and } U_{3,3} \leftarrow 2\frac{1}{6}$$

$$LU \text{ snapshot: } \pi = [2, 3, 3] \quad L = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \quad \text{and} \quad U = \begin{bmatrix} 2 & 2 & -1 \\ 0 & -3 & 1 \\ 0 & 0 & 2\frac{1}{6} \end{bmatrix} .$$

The permutation matrix associated with π is $P = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$ and

$$PA = \begin{bmatrix} 2 & 2 & -1 \\ 2 & -1 & 0 \\ 1 & 3 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 & -1 \\ 0 & -3 & 1 \\ 0 & 0 & 2\frac{1}{6} \end{bmatrix} = L \cdot U.$$

Phase 2: Solve the lower triangular system $L\mathbf{y} = P\mathbf{b}$.

$$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ \frac{1}{2} & -\frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} -3 \\ 3 \\ 1 \end{bmatrix} \Rightarrow y_1 = -3, y_2 = 6, y_3 = 6\frac{1}{2}.$$

Phase 3: Solve the upper triangular system $U\mathbf{x} = \mathbf{y}$.

$$\begin{bmatrix} 2 & 2 & -1 \\ 0 & -3 & 1 \\ 0 & 0 & 2\frac{1}{6} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} -3 \\ 6 \\ 6\frac{1}{2} \end{bmatrix} \Rightarrow x_1 = 1, x_2 = -1, x_3 = 3.$$

38.4 Orthogonalization and QR Decomposition

The process of transforming an arbitrary linear system into a triangular system may also be approached by systematically introducing zeros into the coefficient matrix with unitary transformations: Given a system $\mathbf{Ax} = \mathbf{b}$, find unitary matrices $V_1, V_2, \dots, V_\ell$ such that $V_\ell \dots V_2 V_1 A = T$ is triangular; calculate $\mathbf{y} = V_\ell \dots V_2 V_1 \mathbf{b}$; solve the triangular system $T\mathbf{x} = \mathbf{y}$.

There are two different types of rudimentary unitary transformations that are described here: Householder transformations and Givens transformations.

Definitions:

Let $\mathbf{v} \in \mathbb{C}^n$ be a nonzero vector. The matrix $H = I - \frac{2}{\|\mathbf{v}\|_2^2} \mathbf{v} \mathbf{v}^*$ is called a **Householder transformation** (or **Householder reflector**). In this context, $\mathbf{v}$, is called a **Householder vector**.

For $\theta, \vartheta \in [0, 2\pi)$, let $G(i, j, \theta, \vartheta)$ be an $n \times n$ identity matrix modified so that the (i, i) and (j, j) entries are replaced by $c = \cos(\theta)$, the (i, j) entry is replaced by $s = e^{i\vartheta} \sin(\theta)$, and the (j, i) entry is replaced by $-\bar{s} = -e^{-i\vartheta} \sin(\theta)$:

$$G(i, j, \theta, \vartheta) = \begin{bmatrix} 1 & \cdots & 0 & \cdots & 0 & \cdots & 0 \\ \vdots & \ddots & \vdots & & \vdots & & \vdots \\ 0 & \cdots & c & \cdots & s & \cdots & 0 \\ \vdots & & \vdots & \ddots & \vdots & & \vdots \\ 0 & \cdots & -\bar{s} & \cdots & c & \cdots & 0 \\ \vdots & & \vdots & & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \cdots & 0 & \cdots & 1 \end{bmatrix}.$$

$G(i, j, \theta, \vartheta)$ is called a **Givens transformation** (or **Givens rotation**).

Facts: [GV96]

1. Householder transformations are unitary matrices.
2. Let $\mathbf{a} \in \mathbb{C}^n$ be a nonzero vector. Define $\mathbf{v} = \text{sign}(a_1) \|\mathbf{a}\| \mathbf{e}_1 + \mathbf{a}$ with $\mathbf{e}_1 = [1, 0, \dots, 0]^T \in \mathbb{C}^n$. Then the Householder transformation $H = I - \frac{2}{\|\mathbf{v}\|_2^2} \mathbf{v} \mathbf{v}^*$ satisfies

$$H\mathbf{a} = \alpha \mathbf{e}_1 \quad \text{with} \quad \alpha = -\text{sign}(a_1) \|\mathbf{a}\|.$$

3. [GV96, pp. 210–213]

Algorithm 3: Householder QR Factorization:

Input: matrix $A \in \mathbb{C}^{m \times n}$ with $m \geq n$

Output: the QR factorization $A = QR$, where the upper triangular part of R is stored in the upper triangular part of A

$Q = I_m$

For $k = 1 : n$

$\mathbf{x} = A_{k:m,k}$

$\mathbf{v}_k = \text{sign}(x_1) \|\mathbf{x}\| \mathbf{e}_1 + \mathbf{x}$, where $\mathbf{e}_1 \in \mathbb{C}^{m-k+1}$

$\mathbf{v}_k = \mathbf{v}_k / \|\mathbf{v}_k\|$

$A_{k:m,k:n} = (I_{m-k+1} - 2\mathbf{v}_k \mathbf{v}_k^*) A_{k:m,k:n}$

$Q_{1:k-1,k:m} = Q_{1:k-1,k:m} (I_{m-k+1} - 2\mathbf{v}_k \mathbf{v}_k^*)$

$Q_{k:m,k:m} = Q_{k:m,k:m} (I_{m-k+1} - 2\mathbf{v}_k \mathbf{v}_k^*)$

4. [GV96, p. 215] A Givens rotation is a unitary matrix.
5. [GV96, pp. 216–221] For any scalars $x, y \in \mathbb{C}$, there exists a Givens rotation $G \in \mathbb{C}^{2 \times 2}$ such that

$$G \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} c & s \\ -\bar{s} & c \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} r \\ 0 \end{bmatrix},$$

where c, s , and r can be computed via

- If $y = 0$ (includes the case $x = y = 0$), then $c = 1, s = 0, r = x$.
 - If $x = 0$ (y must be nonzero), then $c = 0, s = \text{sign}(\bar{y}), r = |y|$.
 - If both x and y are nonzero, then $c = |x|/\sqrt{|x|^2 + |y|^2}$,
 $s = \text{sign}(x)\bar{y}/\sqrt{|x|^2 + |y|^2}, r = \text{sign}(x)\sqrt{|x|^2 + |y|^2}$.
6. [GV96, pp. 226–227]

Algorithm 4: Givens QR Factorization

Input: matrix $A \in \mathbb{C}^{m \times n}$ with $m \geq n$

Output: the QR factorization $A = QR$, where the upper triangular part of R is stored in the upper triangular part of A

$Q = I_m$

For $k = 1 : n$

 For $i = k + 1 : m$

$[x, y] = [A_{kk}, A_{ik}]$

 Compute $G = \begin{bmatrix} c & s \\ -\bar{s} & c \end{bmatrix}$ via Fact 5.

$$\begin{bmatrix} A_{k,k:n} \\ A_{i,k:n} \end{bmatrix} = G \begin{bmatrix} A_{k,k:n} \\ A_{i,k:n} \end{bmatrix}$$

$$[Q_{1:m,k}, Q_{1:m,i}] = [Q_{1:m,k}, Q_{1:m,i}]G^*$$

7. [GV96, p. 212] In many applications, it is not necessary to compute Q explicitly in Algorithm 3 and Algorithm 4. See also [TB97, p. 74] for details.
8. [GV96, pp. 225–227] If $A \in \mathbb{R}^{m \times n}$ with $m \geq n$, then the cost of Algorithm 3 without explicitly computing Q is $2n^2(m - n/3)$ flops and the cost of Algorithm 4 without explicitly computing Q is $3n^2(m - n/3)$ flops.
9. [Mey00, p. 349] Algorithm 3 and Algorithm 4 are numerically stable for computing the QR factorization.

Examples:

1. We shall use Givens rotations to transform $A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$ to upper triangular form, as in Algorithm 4.

First, to annihilate the element in position (2,1), we use Fact 5 with $(x, y) = (1, 1)$ and obtain $c = s = 1/\sqrt{2}$; hence:

$$A^{(1)} = G_1 A = \begin{bmatrix} 0.7071 & 0.7071 & 0 \\ -0.7071 & 0.7071 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 1.4142 & 2.1213 \\ 0 & 0.7071 \\ 1 & 3 \end{bmatrix}.$$

Next, to annihilate the element in position (3,1), we use $(x, y) = (1.4142, 1)$ in Fact 5 and get

$$A^{(2)} = G_2 A^{(1)} = \begin{bmatrix} 0.8165 & 0 & 0.5774 \\ 0 & 1 & 0 \\ -0.5774 & 0 & 0.8165 \end{bmatrix} A^{(1)} = \begin{bmatrix} 1.7321 & 3.4641 \\ 0 & 0.7071 \\ 0 & 1.2247 \end{bmatrix}.$$

Finally, we annihilate the element in position (3,2) using $(x, y) = (.7071, 1.2247)$:

$$A^{(3)} = G_3 A^{(2)} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0.5000 & 0.8660 \\ 0 & -0.8660 & 0.5000 \end{bmatrix} A^{(2)} = \begin{bmatrix} 1.7321 & 3.4641 \\ 0 & 1.4142 \\ 0 & 0 \end{bmatrix}.$$

As a result, $R = A^{(3)}$ and $\hat{R}$ consists of the first two rows of $A^{(3)}$. The matrix Q can be computed as the product $G_1^T G_2^T G_3^T$.

2. We shall use Householder reflections to transform A from Example 1 to upper triangular form as in Algorithm 3. First, let $\mathbf{a} = A_{:,1} = [1 \ 1 \ 1]^T$, $\gamma_1 = -\sqrt{3}$, $\tilde{\mathbf{a}} = [-\sqrt{3} \ 0 \ 0]^T$, and $\mathbf{u}_1 = [0.8881 \ 0.3251 \ 0.3251]^T$; then

$$A^{(1)} = (I - 2\mathbf{u}_1 \mathbf{u}_1^T) A = A - \mathbf{u}_1 \overbrace{\begin{bmatrix} 3.0764 & 5.0267 \end{bmatrix}}^{2\mathbf{u}_1^T A} = \begin{bmatrix} -1.7321 & -3.4641 \\ 0 & 0.3660 \\ 0 & 1.3660 \end{bmatrix}.$$

Next, $\gamma_2 = -\|A_{2:3,2}^{(1)}\|_2$, $\mathbf{u}_2 = [0 \ 0.7934 \ 0.6088]^T$, and

$$A^{(2)} = (I - 2\mathbf{u}_2 \mathbf{u}_2^T) A^{(1)} = A^{(1)} - \mathbf{u}_2 \overbrace{\begin{bmatrix} 0 & 2.2439 \end{bmatrix}}^{2\mathbf{u}_2^T A^{(1)}} = \begin{bmatrix} -1.7321 & -3.4641 \\ 0 & -1.4142 \\ 0 & 0 \end{bmatrix}.$$

Note that $R = A^{(2)}$ has changed sign as compared with Example 1. The matrix Q can be computed as $(I - 2\mathbf{u}_1 \mathbf{u}_1^T)(I - 2\mathbf{u}_2 \mathbf{u}_2^T)$. Therefore, we have full information about the transformation if we store the vectors $\mathbf{u}_1$ and $\mathbf{u}_2$.

38.5 Symmetric Factorizations

Real symmetric matrices ($A = A^T$) and their complex analogs, Hermitian matrices (Chapter 8), are specified by roughly half the number of parameters than general $n \times n$ matrices, so one could anticipate benefits that take advantage of this structure.

Definitions:

An $n \times n$ matrix, A , is **Hermitian** if $A = A^* = \bar{A}^T$.

$A \in \mathbb{C}^{n \times n}$ is **positive-definite** if $\mathbf{x}^* A \mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{C}^n$ with $\mathbf{x} \neq 0$.

The **Cholesky decomposition** (or **Cholesky factorization**) of a positive-definite matrix A is $A = G G^*$ with $G \in \mathbb{C}^{n \times n}$ lower triangular and having positive diagonal entries.

Facts: [Hig96], [GV96]

1. A positive-definite matrix is Hermitian. Note that the similar but weaker assertion for a matrix $A \in \mathbb{R}^{n \times n}$ that “ $\mathbf{x}^T A \mathbf{x} > 0$ for all $\mathbf{x} \in \mathbb{R}^n$ with $\mathbf{x} \neq 0$ ” does not imply that $A = A^T$.

2. If $A \in \mathbb{C}^{n \times n}$ is positive-definite, then A has an LU decomposition, $A = LU$, and the diagonal of U , $\{u_{11}, u_{22}, \dots, u_{nn}\}$, has strictly positive entries.
3. If $A \in \mathbb{C}^{n \times n}$ is positive-definite, then the LU decomposition of A satisfies $A = LU$ with $U = D L^*$ and $D = \text{diag}(U)$. Thus, A can be written as $A = LDL^*$ with L unit lower triangular and D diagonal with positive diagonal entries. Furthermore, A has a Cholesky decomposition $A = G G^*$ with $G \in \mathbb{C}^{n \times n}$ lower triangular. Indeed, if

$$\hat{D} = \text{diag}(\{\sqrt{u_{11}}, \sqrt{u_{22}}, \dots, \sqrt{u_{nn}}\})$$

then $\hat{D}\hat{D} = D$ and $G = L \hat{D}$.

4. [GV96, p. 144] The Cholesky decomposition of a positive-definite matrix A can be computed directly:

Algorithm 1: Cholesky decomposition of a positive-definite matrix

Input: $A \in \mathbb{C}^{n \times n}$ positive definite

Output: $G \in \mathbb{C}^{n \times n}$ (lower triangular matrix so that $A = G G^*$)

$G \leftarrow 0 \in \mathbb{C}^{n \times n}$;

For $j = 1$ to n

$\mathbf{v} \leftarrow A_{j:n,j}$

for $k = 1$ to $j - 1$,

$\mathbf{v} \leftarrow \mathbf{v} - \overline{G_{j,k}} G_{j:n,k}$

$G_{j:n,j} \leftarrow \frac{1}{\sqrt{v_1}} \mathbf{v}$

5. Algorithm 1 requires approximately $n^3/3$ floating point arithmetic operations and n floating point square roots to complete (roughly half of what is required for an LU decomposition).
6. If $A \in \mathbb{R}^{n \times n}$ is symmetric and positive-definite and Algorithm 1 runs to completion producing a computed Cholesky factor $\hat{G} \in \mathbb{R}^{n \times n}$, then

$$\hat{G}\hat{G}^T = A + \delta A \quad \text{with} \quad |\delta A| \leq \frac{(n+1)\epsilon}{1 - (n+1)\epsilon} |\hat{G}| |\hat{G}^T|.$$

Furthermore, if an approximate solution, $\hat{\mathbf{x}}$, to $A\mathbf{x} = \mathbf{b}$ is computed by solving the two triangular linear systems $\hat{G}\mathbf{y} = \mathbf{b}$ and $\hat{G}^T\hat{\mathbf{x}} = \mathbf{y}$, and a scaling matrix is defined as $\Delta = \text{diag}(\sqrt{a_{ii}})$, then the scaled error $\Delta(\mathbf{x} - \hat{\mathbf{x}})$ satisfies

$$\frac{\|\Delta(\mathbf{x} - \hat{\mathbf{x}})\|_2}{\|\Delta\mathbf{x}\|_2} \leq \frac{\kappa_2(H)\epsilon}{1 - \kappa_2(H)\epsilon},$$

where $A = \Delta H \Delta$. If $\kappa_2(H) \ll \kappa_2(A)$, then it is quite likely that the entries of $\Delta\hat{\mathbf{x}}$ will have mostly the same magnitude and so the error bound suggests that all entries of the solution will be computed to high relative accuracy.

7. If $A \in \mathbb{C}^{n \times n}$ is Hermitian and has all leading principal submatrices nonsingular, then A has an LU decomposition that can be written as $A = LU = L D L^*$ with L unit lower triangular and D diagonal with real diagonal entries. Furthermore, the number of positive and negative entries of D is equal to the number of positive and negative eigenvalues of A , respectively (the *Sylvester law of inertia*).
8. Note that it may not be prudent to compute the LU (or LDL^T) decomposition of a Hermitian indefinite matrix A without pivoting, yet the use of pivoting will likely eliminate the advantages symmetry might offer. An alternative is a block LDL^T decomposition that incorporates a diagonal pivoting strategy (see [GV96] for details).

Examples:

1. Calculate the Cholesky decomposition of the 3×3 Hilbert matrix,

$$A = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}.$$

Setup:	$G \leftarrow \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$
$j = 1 \text{ step:}$	$\mathbf{v} \leftarrow [1, \frac{1}{2}, \frac{1}{3}]^T$
$G \text{ snapshot:}$	$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & 0 & 0 \\ \frac{1}{3} & 0 & 0 \end{bmatrix}$
$j = 2 \text{ step:}$	$\mathbf{v} \leftarrow \begin{bmatrix} \frac{1}{3} \\ \frac{1}{4} \end{bmatrix} - \frac{1}{2} \begin{bmatrix} \frac{1}{2} \\ \frac{1}{3} \end{bmatrix} = \begin{bmatrix} \frac{1}{12} \\ \frac{1}{12} \end{bmatrix}$
$G \text{ snapshot:}$	$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2\sqrt{3}} & 0 \\ \frac{1}{3} & \frac{1}{2\sqrt{3}} & 0 \end{bmatrix}$
$j = 3 \text{ step:}$	$\mathbf{v} \leftarrow \frac{1}{5} - \left(\frac{1}{3}\right)^2 - \left(\frac{1}{2\sqrt{3}}\right)^2 = \frac{1}{180} = \left(\frac{1}{6\sqrt{5}}\right)^2$
$G \text{ snapshot:}$	$G = \begin{bmatrix} 1 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2\sqrt{3}} & 0 \\ \frac{1}{3} & \frac{1}{2\sqrt{3}} & \frac{1}{6\sqrt{5}} \end{bmatrix}$

References

- [Dem97] J. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
- [GV96] G.H. Golub and C.F. Van Loan. *Matrix Computations*. 3rd ed., Johns Hopkins University Press, Baltimore, MD, 1996.
- [Hig96] N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, 1996.
- [Mey00] C. Meyer. *Matrix Analysis and Applied Linear Algebra*. SIAM, Philadelphia, 2000.
- [StS90] G.W. Stewart and J.-G. Sun. *Matrix Perturbation Theory*. Academic Press, San Diego, CA, 1990.
- [TB97] L.N. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.

39

Least Squares Solution of Linear Systems

Per Christian Hansen
Technical University of Denmark
Hans Bruun Nielsen
Technical University of Denmark

39.1	Basic Concepts	39-1
39.2	Least Squares Data Fitting	39-3
39.3	Geometric and Algebraic Aspects	39-4
39.4	Orthogonal Factorizations	39-5
39.5	Least Squares Algorithms	39-6
39.6	Sensitivity	39-7
39.7	Up- and DOWndating of QR Factorization	39-8
39.8	Damped Least Squares	39-9
39.9	Rank Revealing Decompositions	39-11
	References	39-13

39.1 Basic Concepts

(See Chapter 5 for additional information.)

Definitions:

Given a vector $\mathbf{b} \in \mathbb{R}^m$ and a matrix $A \in \mathbb{R}^{m \times n}$ with $m > n$, the **least squares problem** is to find a vector $\mathbf{x}_0 \in \mathbb{R}^n$ that minimizes the Euclidean length of the difference between $A\mathbf{x}$ and $\mathbf{b}$:

Problem LS: Find $\mathbf{x}_0$ satisfying $\|\mathbf{b} - A\mathbf{x}_0\|_2 = \min \|\mathbf{b} - A\mathbf{x}\|_2$.

Such an $\mathbf{x}_0$ is called a **least squares solution**.

For any vector $\mathbf{x}$ the vector $\mathbf{r} = \mathbf{r}(\mathbf{x}) = \mathbf{b} - A\mathbf{x}$ is the **residual vector**. The residual of a least squares solution is denoted by $\mathbf{r}_0$. The least squares problem is **consistent** if $\mathbf{b} \in \text{range}(A)$.

A **basic solution**, $\mathbf{x}_{0B}$, is a least squares solution with at least $n - \text{rank}(A)$ zero components. The **minimum-norm least squares solution**, $\mathbf{x}_{0M}$, is the least squares solution of minimum Euclidean norm.

In a **weighted least squares problem**, we are also given weights $w_i \geq 0$ for $i = 1, \dots, m$, and the objective is to minimize $\|W(\mathbf{b} - A\mathbf{x})\|_2$, where $W = \text{diag}(w_1, \dots, w_m)$. This is an important special case of the **generalized least squares problem**:

Problem GLS: Let $A\mathbf{x} + B\mathbf{v} = \mathbf{b}$, where $B \in \mathbb{R}^{m \times p}$ with $p \leq m$.
Find $\mathbf{x}_G, \mathbf{v}_G$ such that $\|\mathbf{v}\|_2$ is minimized.

Note that $B\mathbf{v}$ plays the role of the residual vector.

In the **total least squares problem**, we allow for errors both in the vector $\mathbf{b}$ and in the matrix A :

Problem TLS: Let $(A + E)\mathbf{x} + \mathbf{r} = \mathbf{b}$, where $E \in \mathbb{R}^{m \times n}$ with $n \leq m$.

Find $\mathbf{x}_T$ such that $\|(E, \mathbf{r})\|_F$ is minimized.

$\|\cdot\|_F$ denotes the Frobenius norm.

In this chapter, $A_{i,:}$ and $A_{:,j}$ denote the vectors given by the elements in the i th row and j th column of matrix A , respectively. Similarly, $A_{k:l,:}$ (or $A_{:,k:l}$) is the submatrix consisting of rows (or columns) k through l of A .

In the examples in this chapter, the computation is done with about 16 digits accuracy, but the displayed results are rounded to fewer digits.

Facts:

(See, e.g., Chapters 1 and 2 in [Bjo96].)

1. If $m > n = \text{rank}(A)$, then the least squares solution $\mathbf{x}_0$ is analytically equivalent to the solution to the *normal equations* $A^T A \mathbf{x} = A^T \mathbf{b}$.
2. If the least squares problem is consistent, then $\mathbf{r}_0 = \mathbf{0}$.
3. The least squares solution is unique if $m \geq n$ and A has full rank.
4. If the system is underdetermined ($m < n$) or if A is rank deficient, then the solution to Problem LS is not unique. Also, a basic solution is not unique.
5. The minimum-norm least squares solution $\mathbf{x}_{0M}$ is always unique.
6. If $m > n = \text{rank}(A)$, then the least squares solution can be written as $\mathbf{x}_0 = A^\dagger \mathbf{b}$, where the matrix $A^\dagger$ is the Moore–Penrose generalized inverse or pseudoinverse of A . (See Section 5.7.) In general, $A^\dagger$ produces the minimum-norm solution: $\mathbf{x}_{0M} = A^\dagger \mathbf{b}$.
7. If B is nonsingular, then $\mathbf{x}_G$ minimizes $\|B^{-1}(\mathbf{b} - A\mathbf{x})\|_2$.
8. If the covariance matrix for $\mathbf{b}$ has the Cholesky factorization $\text{Cov}(\mathbf{b}) = C^T C$, then $\mathbf{x}_G$ is the *best linear unbiased estimate* (BLUE) in the general linear model with $B = C^T$.
9. If $\text{Cov}(\mathbf{b})$ has full rank, then $\mathbf{x}_G$ is the solution to the least squares problem $\min \|C^{-T}(\mathbf{b} - A\mathbf{x})\|_2$.
10. In particular, if $\text{Cov}(\mathbf{b}) = \sigma^2 I$, then $\mathbf{x}_G = \mathbf{x}_0$ and $\text{Cov}(\mathbf{x}_0) = \sigma^2 (A^T A)^{-1}$.
11. An English translation of the original work on least squares problems by C. F. Gauss is available in [Gau95].

Examples:

1. Consider problem LS with $m = 3$ and $n = 2$:

$$\min \left\| \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 0.75 \\ 1.13 \\ 1.39 \end{bmatrix} \right\|_2.$$

The associated normal equations and the least squares solution are

$$\begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3.27 \\ 7.18 \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} 0.45 \\ 0.32 \end{bmatrix}.$$

The residual vector corresponding to $\mathbf{x}_0$ is

$$\mathbf{r}_0 = \mathbf{r}(\mathbf{x}_0) = \mathbf{b} - A\mathbf{x}_0 = \begin{bmatrix} -0.02 \\ 0.04 \\ -0.02 \end{bmatrix},$$

and $A^T \mathbf{r}_0 = \mathbf{0}$.

2. If we use the weights $w_1 = 10$ and $w_2 = w_3 = 1$, the problem is changed to

$$\min \left\| \begin{bmatrix} 10 & 10 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 7.5 \\ 1.13 \\ 1.39 \end{bmatrix} \right\|_2$$

whose least squares solution and corresponding residual are

$$\mathbf{x}_0 = \begin{bmatrix} 0.41838 \\ 0.33186 \end{bmatrix}, \quad \mathbf{r}_0 = \begin{bmatrix} -0.00024 \\ 0.04790 \\ -0.02395 \end{bmatrix}.$$

Note that the first component of $\mathbf{r}_0$ is reduced when w_1 is increased from 1 to 10.

39.2 Least Squares Data Fitting

Definitions:

Given m data points (t_i, y_i) , $i = 1, \dots, m$, and n linearly independent functions f_j , $j = 1, \dots, n$ (with $m > n$), find the linear combination

$$F(\mathbf{x}, t) = \sum_{j=1}^n x_j f_j(t)$$

that minimizes the sum of squared residuals $y_i - F(\mathbf{x}, t_i)$ at the data points:

$$\min_{\mathbf{x}} \sum_{i=1}^m (y_i - F(\mathbf{x}, t_i))^2.$$

The coefficients x_i are the components of the least squares solution to $\min \|\mathbf{b} - A\mathbf{x}\|_2$, where the columns $A_{:,j}$ of A are samples of f_j at t_i and the elements of $\mathbf{b}$ are the values y_i :

$$A_{ij} = f_j(t_i), \quad b_i = y_i, \quad i = 1, \dots, m \quad j = 1, \dots, n.$$

The solution $F(\mathbf{x}_0, t)$ is said to fit the data in the least squares sense.

Facts:

(See, e.g., Chapter 4 in [Bjo96].)

1. The fit can be made more robust to outliers by solving a weighted least squares problem $\min \|W(\mathbf{b} - A\mathbf{x})\|_2$ with $W = \text{diag}(w_1, \dots, w_m)$, $w_i = \psi(r_i) = \psi(b_i - A_{i,:}\mathbf{x})$; ψ being a convex function. This problem is usually solved by an iteratively reweighted least squares algorithm.
2. In orthogonal distance fitting, instead of minimizing the residuals one minimizes the orthogonal distances between the fitting function F and the data points. Important examples are fitting of a circle, an arc, or an ellipse to data points.

Examples:

1. Given $f_1(t) = 1$ and $f_2(t) = t$, find the least squares fit to the data points $(1, 0.75)$, $(2, 1.13)$, and $(3, 1.39)$. We get the A , $\mathbf{b}$, and $\mathbf{x}_0$ from Example 1 in section 39.1, and

$$F(\mathbf{x}_0, t) = 0.45 + 0.32t.$$

If the third data point is changed to $(3, 13.9)$, then the least squares solution changes to $\mathbf{x}_0 = (-7.890, 6.575)^T$, and the least squares fit becomes $F(\mathbf{x}_0, t) = -7.890 + 6.575t$; this illustrates the sensitivity to outliers.

39.3 Geometric and Algebraic Aspects

Definitions:

The columns of $A \in \mathbb{R}^{m \times n}$ span the **range** of A , while the **nullspace** or **kernel** of A is the set of solutions to the homogeneous system $A\mathbf{x} = \mathbf{0}$:

$$\text{range}(A) = \{\mathbf{z} = A\mathbf{x} \mid \mathbf{x} \in \mathbb{R}^n\}, \quad \ker(A) = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} = \mathbf{0}\}.$$

The four **fundamental subspaces** associated with Problem LS are $\text{range}(A)$, $\ker(A^T)$, $\ker(A)$, and $\text{range}(A^T)$. (See Section 2.4 for more information.)

Facts:

The first three facts can be found in [Str88, Sec. 2.4]; the remaining facts are discussed in [Bjo96, Chap. 1]. p denotes the rank of A : $p = \text{rank}(A)$.

1. If $p = n < m$, then the vector $\mathbf{0}$ of all zeros is the only element in $\ker(A)$.
2. The spaces $\text{range}(A)$ and $\ker(A^T)$ are subspaces of $\mathbb{R}^m$ with dimensions p and $m-p$, respectively. The two spaces are orthogonal complements, i.e., $\mathbf{y}^T \mathbf{z} = 0$ for any pair $(\mathbf{y} \in \text{range}(A), \mathbf{z} \in \ker(A^T))$, and $\text{range}(A) \oplus \ker(A^T) = \mathbb{R}^m$.
3. The spaces $\ker(A)$ and $\text{range}(A^T)$ are subspaces of $\mathbb{R}^n$ with dimensions $n-p$ and p , respectively. The two spaces are orthogonal complements.
4. The least squares residual vector $\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0$ is an element in $\ker(A^T)$. Combining this with the definition of $\mathbf{r}$, we get the so-called **augmented system** associated with Problem LS:

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} \mathbf{r} \\ \mathbf{x} \end{bmatrix} = \begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix}.$$

If $p = n$, then the augmented system is nonsingular and the solution components are $\mathbf{r}_0$ and $\mathbf{x}_0$.

5. The vector $A\mathbf{x}_0$ is the orthogonal projection of $\mathbf{b}$ onto $\text{range}(A)$.
6. The vector $\mathbf{r}_0$ is the orthogonal projection of $\mathbf{b}$ onto $\ker(A^T)$.
7. If $p < n$, then the columns in A can be reordered such that $A\Pi = [\tilde{A} \quad \hat{A}]$, where Π is a permutation matrix and the submatrix $\tilde{A}$ has p columns, $\text{range}(A) = \text{range}(\tilde{A})$. The permutation is not unique.
8. The orthogonal projectors onto $\text{range}(A)$ and $\ker(A)$ are given by $AA^\dagger$ and $I - A^\dagger A$, respectively.

Examples:

1. The Figure 39.1 illustrates Facts 5 and 6 in the case $m = 3, n = p = 2$.

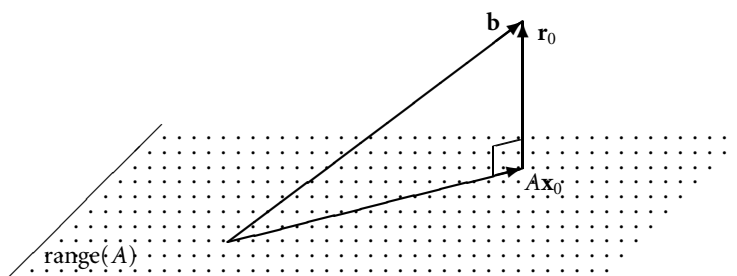


FIGURE 39.1

2. For the problem in Example 1 in Section 39.1, both $\text{range}(A)$ and $\ker(A^T)$ are subspaces of $\mathbb{R}^3$ given by, respectively,

$$\text{range}(A) = \alpha \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} + \beta \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \ker(A^T) = \gamma \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix},$$

with $\alpha, \beta, \gamma \in \mathbb{R}$.

3. Fact 4 can be used to derive the normal equations:

$$\mathbf{r} = \mathbf{b} - A\mathbf{x} \in \ker(A^T) \Rightarrow A^T(\mathbf{b} - A\mathbf{x}) = \mathbf{0}.$$

39.4 Orthogonal Factorizations

(See Section 5.5 and Section 38.4 for additional information on orthogonal factorizations.)

Definitions:

The real matrix Q is **orthogonal** if it is square and satisfies $Q^T Q = I$.

A **QR factorization** of a matrix $A \in \mathbb{R}^{m \times n}$ with $m \geq n$ has the form

$$A = QR = Q \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix} = \hat{Q} \hat{R},$$

where $Q \in \mathbb{R}^{m \times m}$ is orthogonal, $R \in \mathbb{R}^{m \times n}$, $\hat{R} \in \mathbb{R}^{n \times n}$ is upper triangular, and $\hat{Q} = Q_{:,1:n}$. The form $A = \hat{Q} \hat{R}$ is the so-called “reduced” (or “skinny”) QR factorization.

The **singular value decomposition** (SVD) of $A \in \mathbb{R}^{m \times n}$ has the form

$$A = U \Sigma V^T,$$

where $U \in \mathbb{R}^{m \times m}$ and $V \in \mathbb{R}^{n \times n}$ are orthogonal matrices and $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{m \times n}$ has diagonal elements

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r \geq 0, \quad r = \min\{m, n\}.$$

Letting $\mathbf{u}_j$ and $\mathbf{v}_j$ denote the j th column in U and V , respectively, we can write

$$A = \sum_{j=1}^r \sigma_j \mathbf{u}_j \mathbf{v}_j^T.$$

For $k < r$ the matrix $\sum_{j=1}^k \sigma_j \mathbf{u}_j \mathbf{v}_j^T$ is called the **truncated SVD** approximation to A . (See Sections 5.6, 17, and 45 for more information about the singular value decomposition.)

Facts:

Except for Facts 1 and 8 see [Bjo96, Chap. 1]. Also see Section 39.5.

1. [Str88, Chap. 3]. A QR factorization preserves rank: $\text{rank}(R) = \text{rank}(\hat{R}) = \text{rank}(A)$.
2. A QR factorization is not unique, but two factorizations $Q_1 R_1$ and $Q_2 R_2$ always satisfy $\hat{R}_1 = D \hat{R}_2$, where D is diagonal with $D_{ii} = \pm 1$.
3. The triangular factor $\hat{R}$ and the upper triangular Cholesky factor C for the normal equations matrix $A^T A$ always satisfy $\hat{R} = \tilde{D} C$, where $\tilde{D}$ is diagonal with $\tilde{D}_{ii} = \pm 1$.
4. If A has full rank and has the QR factorization $A = QR$, then $\mathbf{x}_0$ can be found by back substitution in the upper triangular system $\hat{R} \mathbf{x} = Q_{:,1:m}^T \mathbf{b}$.

5. If Q has not been saved, then we can use forward and back substitution to solve the seminormal equations: $\widehat{R}^T \widehat{R} \mathbf{x} = A^T \mathbf{b}$. For reasons of numerical stability, this must be followed by one step of iterative refinement; the complete process is called the *corrected seminormal equations* method.
6. Let $\text{rank}(A) = p \leq n \leq m$ and $A = QR$. The columns in $Q_{:,1:p}$ and $Q_{:,p+1:m}$ are orthonormal bases of $\text{range}(A)$ and $\ker(A^T)$, respectively.
7. Let $A = U \Sigma V^T$. Then $p = \text{rank}(A)$ is equal to the number of strictly positive singular values: $\sigma_1 \geq \dots \geq \sigma_p > 0$, $\sigma_{p+1} = \dots = \sigma_{\min\{m,n\}} = 0$. The columns in $U_{:,1:p}$ and $U_{:,p+1:m}$ are orthonormal bases of $\text{range}(A)$ and $\ker(A^T)$, respectively, and the columns in $V_{:,1:p}$ and $V_{:,p+1:n}$ are orthonormal bases of $\text{range}(A^T)$ and $\ker(A)$, respectively.
8. [GV96, Chap. 12]. The TLS solution can be computed as follows: compute the SVD of the coefficient matrix A augmented with the right-hand side $\mathbf{b}$, i.e., $[A, \mathbf{b}] = \widetilde{U} \widetilde{\Sigma} \widetilde{V}^T$. If the smallest singular value $\widetilde{\sigma}_{n+1}$ is simple, and if $\beta = \widetilde{V}_{n+1,n+1} \neq 0$, then

$$\mathbf{x}_T = -\beta^{-1} \widetilde{V}_{1:n,n+1}, \quad E_T = -\widetilde{\sigma}_{n+1} \widetilde{U}_{:,n+1} \widetilde{V}_{1:n,n+1}^T, \quad \mathbf{r}_T = -\widetilde{\sigma}_{n+1} \beta \widetilde{U}_{:,n+1}.$$

Examples:

1. For the problem from Example 1 in Section 39.1, we find $[A, \mathbf{b}] = \widetilde{U} \widetilde{\Sigma} \widetilde{V}^T$ with

$$\widetilde{\Sigma} = \text{diag} \begin{bmatrix} 4.515 \\ 0.6198 \\ 0.0429 \end{bmatrix}, \quad \widetilde{U}_{:,3} = \begin{bmatrix} 0.4248 \\ -0.8107 \\ 0.4029 \end{bmatrix}, \quad \widetilde{V}_{:,3} = \begin{bmatrix} 0.3950 \\ 0.2796 \\ -0.8751 \end{bmatrix}.$$

Thus,

$$\mathbf{x}_T = \begin{bmatrix} 0.4513 \\ 0.3195 \end{bmatrix}, \quad E_T = \begin{bmatrix} -0.0072 & -0.0051 \\ 0.0137 & 0.0097 \\ -0.0068 & -0.0048 \end{bmatrix}, \quad \mathbf{r}_T = \begin{bmatrix} -0.015939 \\ 0.030421 \\ -0.015118 \end{bmatrix}.$$

In Example 1 in section 39.1 we found $\mathbf{x}_0 = [0.4500 \quad 0.3200]^T$. The difference between $\mathbf{x}_0$ and $\mathbf{x}_T$ is small because the problem is almost consistent and A is well conditioned; see Section 39.6. The elements in $\mathbf{r}_T$ are about 80% of the elements in $\mathbf{r}_0$ given in Example 1 in Section 39.1.

39.5 Least Squares Algorithms

Definitions:

By least squares algorithms we mean algorithms for computing the least squares solution efficiently and stably on a computer. The algorithms should take into account the size of the matrix and, if applicable, also its structure.

Facts:

For real systems the following facts can be found in, e.g., [Bjo96], [Bjo04], and [LH95].

1. The algorithm which is least sensitive to the influence of rounding errors is based on the QR factorization of A :
 - (a) Compute the reduced QR factorization $A = \widehat{Q} \widehat{R}$.
 - (b) Compute the vector $\beta = \widehat{Q}^T \mathbf{b}$ (can be computed during the QR factorization algorithm without forming $\widehat{Q}$ explicitly).
 - (c) Compute $\mathbf{x} = \widehat{R}^{-1} \beta$ via back substitution.

The use of this algorithm was first suggested in [Gol65].

2. If A is well conditioned, the normal equations can be used instead:
 - (a) Compute the normal equation system $M = A^T A$ and $\mathbf{d} = A^T \mathbf{b}$.
 - (b) Compute the Cholesky factorization $M = C^T C$ and $\mathbf{y} = C^{-T} \mathbf{d}$ (the vector $\mathbf{y}$ can be computed during the Cholesky algorithm).
 - (c) Compute $\mathbf{x} = C^{-1} \mathbf{y}$ via back substitution.
3. If A or $A^T A$ is a Toeplitz matrix, use an algorithm that utilizes this structure to obtain the computational complexity $O(mn)$.
4. If A is large and sparse, use a sparse QR factorization algorithm that avoids storing the matrix Q . If solving a system with the same A but a different right-hand side, use the corrected seminormal equations.
5. Alternatively, if A is large and sparse, it may be preferable to use the augmented system approach because it may lead to less fill-in. Then a symmetric indefinite solver, such as the LDL^T factorization, must be used, cf. [GV96, Sec. 4.4].
6. If A is large and the matrix-vector multiplications with A and A^T can be computed easily, then use the conjugate gradient algorithm on the normal equations. Several implementations are available; CGLS is the classical formulation; LSQR is more accurate for ill-conditioned matrices.

39.6 Sensitivity

Definitions:

For $A \in \mathbb{R}^{m \times n}$ with $p = \text{rank}(A) \leq \min(m, n)$ the **condition number** for the least squares problem is given by $\kappa(A) = \sigma_1 / \sigma_p$, where $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_p > 0$ are the nonzero singular values of A .

Let $\mathbf{x}_{0M}$ and $\tilde{\mathbf{x}}_{0M}$ denote the minimum-norm least squares solutions to the problems $\min \|\mathbf{b} - A\mathbf{x}\|_2$ and $\min \|\tilde{\mathbf{b}} - \tilde{A}\mathbf{x}\|_2$, the latter being a perturbation of the former. Define the quantities

$$\delta_A = \|\tilde{A} - A\|_2, \quad \delta_b = \|\tilde{\mathbf{b}} - \mathbf{b}\|_2, \quad \eta = \frac{\delta_A}{\sigma_p} = \kappa(A) \frac{\delta_A}{\|A\|_2}.$$

Facts:

(See, e.g., [Bjo96, Sec. 1.4] or [LH95, Chap. 9].)

1. If $\eta < 1$, then the rank is not changed by the perturbation $\text{rank}(\tilde{A}) = \text{rank}(A)$.
2. If $\eta < 1$, then the relative perturbation in $\tilde{\mathbf{x}}_{0M}$ is bounded as

$$\frac{\|\tilde{\mathbf{x}}_{0M} - \mathbf{x}_{0M}\|_2}{\|\mathbf{x}_{0M}\|_2} \leq \frac{\kappa(A)}{1 - \eta} \left(\frac{\delta_A}{\|A\|_2} + \frac{\delta_b + \eta \|\mathbf{r}_0\|_2}{\|A\|_2 \|\mathbf{x}_{0M}\|_2} \right) + \eta, \quad \mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_{0M}.$$

If $\text{rank}(A) = n$, then the last term η is omitted. If the problem is consistent, i.e., $\mathbf{r}_0 = \mathbf{0}$, then the relative error can be expected to grow linearly with κ . For $\mathbf{r}_0 \neq \mathbf{0}$ the contribution $\eta \|\mathbf{r}_0\|_2$ and the definition of η show that the relative error may grow as $\kappa(A)^2$.

3. The condition number for the normal equations matrix is $\kappa(A^T A) = \kappa(A)^2$. Due to the finite computer precision, information may be lost when the normal equations are formed; see, e.g., [Bjo96, Sec. 2.2] and Example 2 below.
4. *Component-wise perturbation theory* applies when component-wise perturbation bounds are available for the errors in A and $\mathbf{b}$; if

$$|\tilde{A} - A| \leq \epsilon |A| \quad \text{and} \quad |\tilde{\mathbf{b}} - \mathbf{b}| \leq \epsilon |\mathbf{b}|,$$

where the absolute values and inequalities are interpreted componentwise, then

$$|\tilde{\mathbf{x}} - \mathbf{x}| = \epsilon |A^\dagger| (|\mathbf{b}| + |A| |\mathbf{x}|) + \epsilon |(A^T A)^{-1}| |E^T| |\mathbf{r}|$$

and

$$\|\tilde{\mathbf{x}} - \mathbf{x}\|_\infty \leq \epsilon \left\| |A^\dagger| (|A| |\mathbf{x}| + |\mathbf{b}|) \right\|_\infty + \epsilon \left\| |(A^T A)^{-1}| |A^T| |\mathbf{r}| \right\|_\infty.$$

See [Bjo96, Secs. 1.4.5–6] for further references about component-wise perturbation analysis and *a posteriori* error estimation.

Examples:

1. We consider the problem from Example 1 in Section 39.1 and two perturbed versions of it,

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 0.75 \\ 1.13 \\ 1.39 \end{bmatrix}, \quad \tilde{A} = \begin{bmatrix} 0.8 & 1.1 \\ 0.95 & 2 \\ 1.1 & 2.95 \end{bmatrix}, \quad \tilde{\mathbf{b}} = \begin{bmatrix} 0.79 \\ 1.23 \\ 1.30 \end{bmatrix}.$$

The matrix has full rank, so the least squares solution is unique (and equal to the minimum-norm least squares solution). The vectors

$$\mathbf{x}_0 = \begin{bmatrix} 0.4500 \\ 0.3200 \end{bmatrix}, \quad \tilde{\mathbf{x}}_1 = \begin{bmatrix} 0.5967 \\ 0.2550 \end{bmatrix}, \quad \tilde{\mathbf{x}}_2 = \begin{bmatrix} 0.8935 \\ 0.1280 \end{bmatrix}$$

are the minimizers of $\|\mathbf{b} - A\mathbf{x}\|_2$, $\|\tilde{\mathbf{b}} - A\mathbf{x}\|_2$, and $\|\tilde{\mathbf{b}} - \tilde{A}\mathbf{x}\|_2$, respectively.

The matrix has condition number $\kappa(A) = 6.793$, and $\eta = \kappa(A) * \|\tilde{A} - A\|_2 / \|A\|_2 = 0.4230 < 1$. The relative errors and their upper bounds are

$$\frac{\|\tilde{\mathbf{x}}_1 - \mathbf{x}_0\|_2}{\|\mathbf{x}_0\|_2} = 0.291 \leq \kappa(A) \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|_2}{\|A\|_2 \|\mathbf{x}_0\|_2} = 0.423$$

$$\frac{\|\tilde{\mathbf{x}}_2 - \mathbf{x}_0\|_2}{\|\mathbf{x}_0\|_2} = 0.875 \leq \frac{1}{1 - \eta} \left(\eta + \kappa(A) \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|_2 + \eta \|\mathbf{b} - A\mathbf{x}_0\|_2}{\|A\|_2 \|\mathbf{x}_0\|_2} \right) = 1.575.$$

2. Consider the following matrix A and the corresponding normal equation matrix:

$$A = \begin{bmatrix} 1 & 1 \\ \delta & 0 \\ 0 & \delta \end{bmatrix}, \quad A^T A = \begin{bmatrix} 1 + \delta^2 & 1 \\ 1 & 1 + \delta^2 \end{bmatrix}.$$

If $|\delta| \leq \sqrt{\epsilon}$ (where ϵ is the machine precision), then the quantity $1 + \delta^2$ is represented by 1 on the computer and, therefore, the computed $A^T A$ is singular. If we use Householder transformations to compute the QR factorization, we get $R = \begin{bmatrix} -1 & -1 \\ 0 & \delta\sqrt{2} \end{bmatrix}$, so information about δ is preserved.

39.7 Up- and Downdating of QR Factorization

Definitions:

Given $A \in \mathbb{R}^{m \times n}$ with $m > n$ and its QR factorization, as well as a row vector $\mathbf{a}^T$ with $\mathbf{a} \in \mathbb{R}^n$, **updating** of the factorization means computing the QR factorization of the augmented matrix $\begin{bmatrix} A \\ \mathbf{a}^T \end{bmatrix} = \tilde{A} = \tilde{Q}\tilde{R}$ from the QR factors of A . Similarly, **downdating** means computing the QR factors of A from those of $\tilde{A}$. Up- and updating algorithms require only $O(mn)$ flops (compared to the $O(mn^2)$ flops of recomputing the QR factors).

Facts:

The following facts can be found in Chapter 3 of [Bjo96].

1. The matrix R can be updated to $\tilde{R}$ without knowledge of Q by means a sequence of n Givens transformations $G_1, \dots, G_n$.
2. The updating of Q to $\tilde{Q}$ then takes the form

$$\tilde{Q} = \begin{bmatrix} Q & 0 \\ 0 & 1 \end{bmatrix} G_1^T \cdots G_n^T.$$

3. Downdating of $\tilde{R}$ to R requires the first row $\tilde{Q}_{1,:}$ of $\tilde{Q}$. Let $\tilde{G}_1, \dots, \tilde{G}_{m-n}$ be Givens rotations such that $\hat{R} = G_{m-n} \cdots G_1 (\tilde{R} \tilde{Q}_{1,:}^T)$ is upper triangular. Then $R = \hat{R}_{2:n+1,2:n}$.
4. If $\tilde{Q}$ is not available, then its first row $\tilde{Q}_{1,:}$ can be computed by the LINPACK/ Saunders algorithm or via hyperbolic rotations. If A is available, the corrected seminormal equations provide a more accurate algorithm.
5. Up- and downdating algorithms are also available for the cases where a column is appended to or deleted from A .
6. Up- and downdating of the Cholesky factor under a rank-one modification $\tilde{M} = M \pm \mathbf{a}\mathbf{a}^T$ is analytically equivalent to updating R from the QR factorization. In the downdating case the matrix $\tilde{M}$ must be positive (semi)definite.

Examples:

1. Let

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} = QR = \begin{bmatrix} 0.5774 & -0.7071 & -0.4082 \\ 0.5774 & 0 & 0.8165 \\ 0.5774 & 0.7071 & -0.4082 \end{bmatrix} \begin{bmatrix} 1.732 & 3.464 \\ 0 & 1.414 \\ 0 & 0 \end{bmatrix}.$$

If $\mathbf{a}^T = (1 \ 4)$, then

$$\tilde{A} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} = \tilde{Q} \tilde{R} \quad \text{with} \quad \tilde{R} = \begin{bmatrix} 2 & 5 \\ 0 & 2.236 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

The updated factor $\tilde{R}$ is computed by augmenting R with $\mathbf{a}^T$ and applying two left Givens rotations G_1 and G_2 to row pairs (1,4) and (2,4), respectively:

$$\begin{bmatrix} R \\ \mathbf{a}^T \end{bmatrix} = \begin{bmatrix} 1.732 & 3.464 \\ 0 & 1.414 \\ 0 & 0 \\ 1 & 4 \end{bmatrix} \xrightarrow{G_1} \begin{bmatrix} 2 & 5 \\ 0 & 1.414 \\ 0 & 0 \\ 0 & 1.732 \end{bmatrix} \xrightarrow{G_2} \begin{bmatrix} 2 & 5 \\ 0 & 2.236 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}.$$

39.8 Damped Least Squares

Definitions:

The **damped least squares** solution is the solution to the problem $(A^T A + \alpha I)\mathbf{x} = A^T \mathbf{b}$, where $\alpha > 0$ and A and $\mathbf{b}$ are real. Damped least squares is also known as **ridge regression** and **Tikhonov** (or **Phillips**) **regularization**.

Facts:

(See, e.g., [Han98] for further details.)

1. The two formulations

$$\min\{\|\mathbf{b} - A\mathbf{x}\|_2^2 + \alpha\|\mathbf{x}\|_2^2\} \quad \text{and} \quad \min\left\|\begin{bmatrix} \mathbf{b} \\ \mathbf{0} \end{bmatrix} - \begin{bmatrix} A \\ \sqrt{\alpha}I \end{bmatrix}\mathbf{x}\right\|_2$$

are analytically equivalent.

2. The damping (controlled by the parameter α) reduces the variance of the solution, at the cost of introducing bias.
3. If $\text{Cov}(\mathbf{b}) = I$, then the covariance matrix for the damped least squares solution $\mathbf{x}_\alpha$ is

$$\text{Cov}(\mathbf{x}_\alpha) = (A^T A + \alpha I)^{-1} A^T A (A^T A + \alpha I)^{-1} = V \Sigma^2 (\Sigma^2 + \alpha I)^{-2} V^T,$$

where Σ and V are from the SVD of A . Hence,

$$\|\text{Cov}(\mathbf{x}_\alpha)\|_2 = \max_i \{\sigma_i^2 / (\sigma_i^2 + \alpha)^2\} \leq (4\alpha)^{-1},$$

while $\|\text{Cov}(\mathbf{x}_0)\|_2 = \|(A^T A)^{-1}\|_2 = \sigma_n^{-2}$, which can be much larger.

4. The expected value of $\mathbf{x}_\alpha$ is

$$\mathcal{E}(\mathbf{x}_\alpha) = (A^T A + \alpha I)^{-1} A^T A \mathbf{x}_0,$$

which introduces a bias because $\mathcal{E}(\mathbf{x}_\alpha) \neq \mathcal{E}(\mathbf{x}_0)$ when $\alpha > 0$.

5. The damped least squares problem can take the more general form

$$\min\{\|\mathbf{b} - A\mathbf{x}\|_2^2 + \alpha\|B\mathbf{x}\|_2^2\} \quad \Leftrightarrow \quad (A^T A + \alpha B^T B) \mathbf{x} = A^T \mathbf{b},$$

where $\|B \cdot\|_2$ defines a (semi)norm. The solution to this problem is unique when the nullspaces of A and B intersect trivially.

6. [Han98, Chap. 7]. Some algorithms for computing α are the discrepancy principle, generalized cross validation, and the L-curve criterion.

Examples:

1. Let $\delta = 10^{-5}$ and consider the matrix and vectors

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 + \delta \\ 1 & 1 + 2\delta \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \mathbf{b} = A \mathbf{x}_0, \quad \tilde{\mathbf{b}} = \mathbf{b} + \begin{bmatrix} 0 \\ 0 \\ \delta \end{bmatrix}.$$

Obviously $\mathbf{x}_0$ is the least squares solution to $A\mathbf{x} + \mathbf{r} = \mathbf{b}$ with $\mathbf{r}_0 = \mathbf{0}$. The minimizer of $\|\tilde{\mathbf{b}} - A\mathbf{x}\|_2$ is $\tilde{\mathbf{x}}_0 = (0.5 \ 1.5)^T$, showing that for this problem the least squares solution is very sensitive to perturbations (the condition number is $\kappa(A) = 2.4 \cdot 10^5$). Using $\alpha = 10^{-8}$, we obtain the damped least squares solutions

$$\mathbf{x}_\alpha = \begin{bmatrix} 0.999995 \\ 1.000005 \end{bmatrix} \quad \text{and} \quad \tilde{\mathbf{x}}_\alpha = \begin{bmatrix} 0.995 \\ 1.005 \end{bmatrix}.$$

Comparing the damped and the undamped least squares solutions $\tilde{\mathbf{x}}_\alpha$ and $\tilde{\mathbf{x}}_0$ to the perturbed least squares problem, we see that $\tilde{\mathbf{x}}_\alpha$ is a better approximation to the unperturbed solution $\mathbf{x}_0$ than $\tilde{\mathbf{x}}_0$.

39.9 Rank Revealing Decompositions

Definitions:

A **rank revealing decomposition** is a two-sided orthogonal decomposition of the form

$$A = U \hat{R} V^T = U \begin{bmatrix} R \\ 0 \end{bmatrix} V^T,$$

where U and V are orthogonal, and R is upper triangular and reveals the (numerical) rank of A in the size of its diagonal elements.

The **numerical rank** k_τ of A , with respect to the threshold τ , is defined as

$$k_\tau = \min \text{rank}(A + E) \quad \text{subject to} \quad \|E\|_2 \leq \tau.$$

Facts:

(See, e.g., [Bjo96, Sec. 1.7.3–6], [Han98, Sec. 2.2], and [Ste98, Chap. 5].)

1. [GV96, p. 73] The numerical rank k_τ is equal to the number of singular values greater than τ , i.e., $\sigma_{k_\tau} > \tau \geq \sigma_{k_\tau+1}$.
2. The singular value decomposition is rank revealing with the middle matrix $R = \Sigma$. The SVD is difficult to update.
3. If A is exactly rank deficient with $\text{rank}(A) = p$, then there always exists a **pivoted QR factorization** $A \Pi = Q \hat{R}$ with $\hat{R}$ of the form

$$\hat{R} = \begin{bmatrix} R_{11} & R_{12} \\ 0 & 0 \end{bmatrix}, \quad R_{11} \in \mathbb{R}^{p \times p}, \quad \text{rank}(\hat{R}) = \text{rank}(R_{11}) = p,$$

and a **complete orthogonal decomposition** of the form $A = U \hat{R} V^T$, where

$$\hat{R} = \begin{bmatrix} R_{11} & 0 \\ 0 & 0 \end{bmatrix}, \quad R_{11} \in \mathbb{R}^{p \times p}, \quad \text{rank}(\hat{R}) = \text{rank}(R_{11}) = p.$$

The pseudoinverse of A is $A^\dagger = V_{:,1:p} R_{11}^{-1} U_{:,1:p}^T$.

4. A basic solution can be computed from the pivoted QR factorization as $\mathbf{x}_{0B} = \Pi R_{11}^{-1} Q_{:,1:p}^T \mathbf{b}$. The minimum-norm least squares solution is given in terms of the complete orthogonal decomposition as $\mathbf{x}_{0M} = V_{:,1:p} R_{11}^{-1} U_{:,1:p}^T \mathbf{b}$.
5. The **rank revealing QR (RRQR) decomposition** is a pivoted QR factorization $A \Pi = Q \hat{R}$ such that

$$\sigma_i / c_i \leq \sigma_i(R_{1:i,1:i}) \leq \sigma_i \leq \|R_{1:i,1:i}\|_2 \leq c_i \sigma_i, \quad i = 1, \dots, n,$$

where σ_i is the i th singular value of A ,

$$c_i = \sqrt{i(n-i) + \min(i, n-i)},$$

and $\sigma_i(R_{1:i,1:i})$ denotes the smallest singular value of $R_{1:i,1:i}$. The RRQR factorization can be used to estimate the numerical rank k_τ . The RRQR factorization is not unique.

6. The **URV decomposition** is a two-sided orthogonal decomposition $A = U \hat{R} V^T$, such that, for $k = 1, \dots, n$,

$$\sigma_i \check{c}_k \leq \sigma_i(R_{1:i,1:i}) \leq \sigma_i, \quad i = 1, \dots, k$$

and

$$\sigma_i \leq \sigma_{i-k}(R_{k+1:n,k+1:n}) \leq \sigma_i / \check{c}_k, \quad i = k+1, \dots, n,$$

where

$$\check{c}_k = \left(1 - \frac{\|R_{k+1:n,k+1:n}\|_2^2}{\sigma_k(R_{1:i,1:i})^2 - \|R_{k+1:n,k+1:n}\|_2^2} \right)^{1/2}.$$

There is also a ULV decomposition with a lower triangular middle matrix; both can be used to estimate the numerical rank of A .

7. The RRQR, URV, and ULV decompositions can be updated, at slightly more cost than the QR factorization.

Examples:

1. The rank of A is revealed by the zero element in the (3,3) position of R :

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 3 & 4 \\ 3 & 4 & 5 \\ 4 & 5 & 6 \end{bmatrix} = Q R \quad \text{with} \quad R = \begin{bmatrix} 5.477 & 7.303 & 9.129 \\ 0 & 0.816 & 1.633 \\ 0 & 0 & 0 \end{bmatrix}.$$

Here the QR factorization is rank revealing ($U = Q$ and $V = I$).

2. Pivoting must be used to ensure that a QR factorization is rank revealing. The “standard column pivoting” often works well in connection with Householder transformations; here the pivot column in each stage is chosen to maximize the norm of the leading column of the submatrix $A_{k:m,k:n}^{(k)}$ to be reduced. Example:

$$A = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix},$$

$$A^{(1)} = H_1 A = \begin{bmatrix} -4.4721 & -2.2361 & -2.6833 \\ 0 & 0 & 0.8944 \\ 0 & 0 & 1 \end{bmatrix},$$

$$A^{(1)} \Pi = \begin{bmatrix} -4.4721 & -2.6833 & -2.2361 \\ 0 & 0.8944 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \Pi = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix},$$

$$\hat{R} = A^{(2)} = H_2 A^{(1)} \Pi = \begin{bmatrix} -4.4721 & -2.6833 & -2.2361 \\ 0 & -1.3416 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

3. The standard column pivoting strategy is not guaranteed to reveal the numerical rank; hence, the development of the RRQR and URV decompositions.

References

- [Bjo96] Å. Björck. *Numerical Methods for Least Squares Problems*. SIAM, Philadelphia, 1996.
- [Bjo04] Å. Björck. The calculation of linear least squares problems. *Acta Numerica* (2004):1–53, Cambridge University Press, Cambridge, 2004.
- [Gau95] C.F. Gauss. *Theory of the Combination of Observations Least Subject to Errors*. (Translated by G. W. Stewart.) SIAM, Philadelphia, 1995.
- [Gol65] G.H. Golub. Numerical methods for solving least squares problems. *Numer. Math.*, 7:206–216, 1965.
- [GV96] G.H. Golub and C.F. Van Loan. *Matrix Computations*, 3rd ed., Johns Hopkins University Press, Baltimore, MD, 1996.
- [Han98] P.C. Hansen. *Rank-Deficient and Discrete Ill-Posed Problems: Numerical Aspects of Linear Inversion*. SIAM, Philadelphia, 1998.
- [LH95] C.L. Lawson and R.J. Hanson. *Solving Least Squares Problems*. Classics in Applied Mathematics, SIAM, Philadelphia, 1995.
- [Ste98] G.W. Stewart. *Matrix Algorithms Volume I: Basic Decompositions*. SIAM, Philadelphia, 1998.
- [Str88] G. Strang. *Linear Algebra and Its Applications*, 3rd ed., Saunders College Publishing, Fort Worth, TX, 1988.

40

Sparse Matrix Methods

	40.1	Introduction	40-1
	40.2	Sparse Matrices	40-2
	40.3	Sparse Matrix Factorizations	40-4
	40.4	Modeling and Analyzing Fill	40-10
	40.5	Effect of Reorderings	40-14
Esmond G. Ng		References	40-18
<i>Lawrence Berkeley National Laboratory</i>			

40.1 Introduction

Let A be an n by n nonsingular matrix and $\mathbf{b}$ be an n -vector. As discussed in Chapter 38, Matrix Factorizations and Direct Solution of Linear Systems, the solution of the system of linear equations $A\mathbf{x} = \mathbf{b}$ using Gaussian elimination requires $O(n^3)$ operations, which typically include additions, subtractions, multiplications, and divisions. The solution also requires $O(n^2)$ words of storage. The computational complexity is based on the assumption that every element of the matrix has to be stored and operated on. However, linear systems that arise in many scientific and engineering applications can be large; that is, n can be large. It is not uncommon for n to be over hundreds of thousands or even millions. Fortunately, for these linear systems, it is often the case that most of the elements in the matrix A will be zero.

Following is a simple example that illustrates where the zero elements come from. Consider the Laplace equation defined on a unit square:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0.$$

Assume that u is known along the boundary. Suppose the square domain is discretized into a $(k+2)$ by $(k+2)$ mesh with evenly spaced mesh points, as shown in Figure 40.1. Also suppose that the mesh points are labeled from 0 to $k+1$ in the x and y directions. For $0 \leq i \leq k+1$, let the variables in the x direction be denoted by x_i . Similarly, for $0 \leq j \leq k+1$, let the variables in the y direction be denoted by y_j . The solution at (x_i, y_j) will be denoted by $u_{i,j} = u(x_i, y_j)$. To solve the Laplace equation numerically, the partial derivatives at (x_i, y_j) will be approximated, for example, using second-order centered difference approximations:

$$\begin{aligned}\left. \frac{\partial^2 u}{\partial x^2} \right|_{(x_i, y_j)} &\approx \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h^2}, \\ \left. \frac{\partial^2 u}{\partial y^2} \right|_{(x_i, y_j)} &\approx \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h^2},\end{aligned}$$

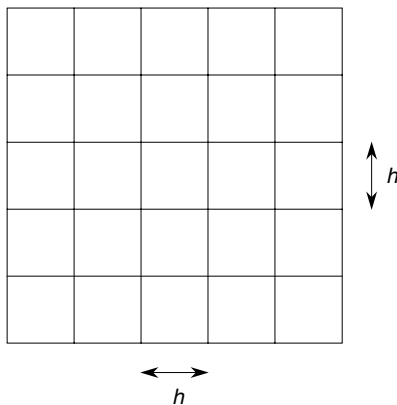


FIGURE 40.1 Discretization of a unit square.

where $h = \frac{1}{k+1}$ is the spacing between two mesh points in each direction. Here, it is assumed that $u_{0,j}$, $u_{k+1,j}$, $u_{i,0}$, $u_{i,k+1}$ are given by the boundary condition, for $1 \leq i, j \leq k$. Using the difference approximations, the Laplace equation at each mesh point (x_i, y_j) , $1 \leq i, j \leq k$, is approximated by the following linear equation:

$$u_{i-1,j} + u_{i,j-1} - 4u_{i,j} + u_{i+1,j} + u_{i,j+1} = 0, \quad \text{for } 1 \leq i, j \leq k.$$

This leads to a system of k^2 by k^2 linear equations in k^2 unknowns. The solution to the linear system provides the approximate solution $u_{i,j}$, $1 \leq i, j \leq k$, at the mesh points. Note that each equation has at most five unknowns. Thus, the coefficient matrix of the linear system, which is k^2 by k^2 and has k^4 elements, has at most $5k^2$ nonzero elements. It is therefore crucial, for the purpose of efficiency (both in terms of operations and storage), to take advantage of the zero elements as much as possible when solving the linear system. The goal is to compute the solution without storing and operating on most of the zero elements of the matrix. This chapter will discuss some of techniques for exploiting the zero elements in Gaussian elimination.

Throughout this chapter, the matrices are assumed to be real. However, most of the discussions are also applicable to complex matrices, with the exception of those on real symmetric positive definite matrices. The discussions related to real symmetric positive definite matrices are applicable to Hermitian positive definite matrices (which are complex but not symmetric).

40.2 Sparse Matrices

Definitions:

A matrix A is **sparse** if substantial savings in either operations or storage can be achieved when the zero elements of A are exploited during the application of Gaussian elimination to A .

The number of nonzero elements in a matrix A is denoted by $\text{nmz}(A)$.

Facts: [DER89], [GL81]

1. Let A be an n by n sparse matrix. It often takes much less than n^2 words to store the nonzero elements in A .
2. Let T be an n by n sparse triangular matrix. The number of operations required to solve the triangular system of linear equations $T\mathbf{x} = \mathbf{b}$ is $O(\text{nmz}(T))$.

Examples:

1. A tridiagonal matrix $T = [t_{i,j}]$ has the form

$$T = \begin{bmatrix} t_{1,1} & t_{1,2} & & & & \\ t_{2,1} & t_{2,2} & t_{2,3} & & & \\ & t_{3,2} & t_{3,3} & t_{3,4} & & \\ & & \ddots & \ddots & \ddots & \\ & & & t_{n-2,n-3} & t_{n-2,n-2} & t_{n-2,n-1} \\ & & & & t_{n-1,n-2} & t_{n-1,n-1} & t_{n-1,n} \\ & & & & & t_{n,n-1} & t_{n,n} \end{bmatrix},$$

where $t_{i,i} \neq 0$ ($1 \leq i \leq n$), and $t_{i,i+1} \neq 0$, and $t_{i+1,i} \neq 0$, ($1 \leq i \leq n-1$). The matrix T is an example of a sparse matrix. If Gaussian elimination is applied to T with partial pivoting for numerical stability, $t_{i,i+2}$, $1 \leq i \leq n-2$, may become nonzero. Thus, in the worst case, there will be at most $5n$ nonzero elements in the triangular factorization (counting the 1s on the diagonal of the lower triangular factor). The number of operations required in Gaussian elimination is at most $5n$.

2. Typically only the nonzero elements of a sparse matrix have to be stored. One of the common ways to store a sparse matrix A is the *compressed column storage* (CCS) scheme. The nonzero elements are stored in an array (e.g., VAL) column by column, along with an integer array (e.g., IND) that stores the corresponding row subscripts. Another integer array (e.g., COLPTR) will be used to provide the index k where VAL(k) contains the first nonzero element in column k of A . Suppose that A is given below:

$$A = \begin{bmatrix} 11 & 0 & 16 & 19 & 22 \\ 0 & 13 & 0 & 0 & 23 \\ 12 & 14 & 17 & 0 & 0 \\ 0 & 0 & 18 & 20 & 24 \\ 0 & 15 & 0 & 21 & 25 \end{bmatrix}.$$

The CCS scheme for storing the nonzero elements is depicted in Figure 40.2. For example, the nonzero elements in column 4 and the corresponding row subscripts can be found in VAL(s) and IND(s), where $s = \text{COLPTR}(4)$, $\text{COLPTR}(4) + 1$, $\text{COLPTR}(4) + 2, \dots, \text{COLPTR}(5) - 1$. Note that, for this example, $\text{COLPTR}(6) = 16$, which is one more than the number of nonzero elements in A . This is used to indicate the end of the set of nonzero elements.

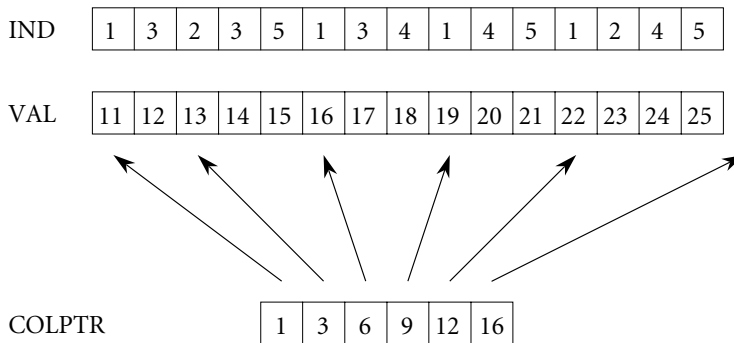


FIGURE 40.2 An example of a compressed column storage scheme.

3. The *compressed row storage* (CRS) scheme is another possibility of storing the nonzero elements of a sparse matrix. It is very similar to the CCS scheme, except that the nonzero elements are stored by rows.
4. Let $A = [a_{i,j}]$ be an n by n sparse matrix. Let $\mathbf{b} = [b_i]$ and $\mathbf{c} = [c_i]$ be n -vectors. The following algorithm computes the product $\mathbf{c} = A\mathbf{b}$, with the assumption that the nonzero elements of A are stored using the CRS scheme.

```

for  $i = 1, 2, \dots, n$  do
     $c_i \leftarrow 0$ 
    for  $s = \text{ROWPTR}(i), \text{ROWPTR}(i) + 1, \dots, \text{ROWPTR}(s + 1) - 1$  do
         $c_i \leftarrow c_i + \text{VAL}(s)b_{\text{IND}(s)}.$ 

```

5. Let $A = [a_{i,j}]$ be an n by n sparse matrix. Let $\mathbf{b} = [b_i]$ and $\mathbf{c} = [c_i]$ be n -vectors. The following algorithm computes the product $\mathbf{c} = A\mathbf{b}$, with the assumption that the nonzero elements of A are stored using the CCS scheme.

```

for  $i = 1, 2, \dots, n$  do
     $c_i \leftarrow 0$ 
for  $i = 1, 2, \dots, n$  do
    for  $s = \text{COLPTR}(i), \text{COLPTR}(i) + 1, \dots, \text{COLPTR}(s + 1) - 1$  do
         $c_{\text{IND}(s)} \leftarrow c_{\text{IND}(s)} + \text{VAL}(s)b_i.$ 

```

40.3 Sparse Matrix Factorizations

Mathematically, computing a triangular factorization of a sparse matrix using Gaussian elimination is really no different from that of a dense matrix. However, the two are very different algorithmically. Sparse triangular factorizations can be quite complicated because of the need to preserve the zero elements as much as possible.

Definitions:

Let A be a sparse matrix. An element of a sparse matrix A is a **fill element** if it is zero in A but becomes nonzero during Gaussian elimination.

The **sparsity structure** of a matrix $A = [a_{i,j}]$ refers to the set $\text{Struct}(A) = \{(i, j) : a_{i,j} \neq 0\}$.

Consider applying Gaussian elimination to a matrix A with row and column pivoting:

$$A = P_1 M_1 P_2 M_2 \cdots P_{n-1} M_{n-1} U Q_{n-1} \cdots Q_2 Q_1,$$

where P_i and Q_i ($1 \leq i \leq n - 1$), are, respectively, the row and column permutations due to pivoting, M_i ($1 \leq i \leq n - 1$) is a Gauss transformation (see Chapter 38), and U is the upper triangular factor. Let $L = M_1 + M_2 + \cdots + M_{n-1} - (n - 2)I$, where I is the identity matrix. Note that L is a lower triangular matrix. The matrix $F = L + U - I$ is referred to as the **fill matrix**.

The matrix A is said to have a **zero-free diagonal** if all the diagonal elements of A are nonzero. The zero-free diagonal is also known as a **maximum transversal**.

No **exact numerical cancellation** between two numbers u and v means that $u + v$ (or $u - v$) is nonzero regardless of the values of u and v .

Facts: [DER89], [GL81]

In the following discussion, A is a sparse nonsingular matrix and F is its fill matrix.

1. [Duf81], [DW88] There exists a (row) permutation matrix P_r such that $P_r A$ has a zero-free diagonal. Similarly, there exists a (column) permutation matrix P_c such that AP_c has a zero-free diagonal.

2. It is often true that there are more nonzero elements in F than in A .
3. The sparsity structure of F depends on the sparsity structure of A , as well as the pivot sequence needed to maintain numerical stability in Gaussian elimination. This means that $\text{Struct}(F)$ is known only during numerical factorization. As a result, the storage scheme cannot be created in advance to accommodate the fill elements that occur during Gaussian elimination.
4. If A is symmetric and positive definite, then pivoting for numerical stability is not needed during Gaussian elimination. Assume that exact numerical cancellations do not occur during Gaussian elimination. Then $\text{Struct}(A) \subseteq \text{Struct}(F)$.
5. Let A be symmetric and positive definite. Assume that exact numerical cancellations do not occur during Gaussian elimination. Then $\text{Struct}(F)$ is determined solely by $\text{Struct}(A)$. This implies that $\text{Struct}(F)$ can be computed before any numerical factorization proceeds. Knowing $\text{Struct}(F)$ in advance allows a storage scheme to be set up prior to numerical factorization.
6. [GN85] Suppose that A is a nonsymmetric matrix. Consider applying Gaussian elimination to a matrix A with partial pivoting:

$$A = P_1 M_1 P_2 M_2 \cdots P_{n-1} M_{n-1} U,$$

where P_i ($1 \leq i \leq n-1$), is a row permutation due to pivoting, M_i ($1 \leq i \leq n-1$) is a Gauss transformation, and U is the upper triangular factor. Let F denote the corresponding fill matrix; that is, $F = M_1 + M_2 + \cdots + M_{n-1} + U - (n-1)I$. The matrix product $A^T A$ is symmetric and positive definite and, hence, has a Cholesky factorization $A^T A = L_C L_C^T$. Assume that A has a zero-free diagonal. Then $\text{Struct}(F) \subseteq \text{Struct}(L_C + L_C^T)$. This result holds for *every* legitimate sequence of pivots $\{P_1, P_2, \dots, P_{n-1}\}$. Thus, $\text{Struct}(L_C)$ and $\text{Struct}(L_C^T)$ can serve as *upper bounds* on $\text{Struct}(L)$ and $\text{Struct}(U)$, respectively. When A is irreducible, then $\text{Struct}(L_C^T)$ is a *tight* bound on $\text{Struct}(U)$: for a given $(i, j) \in \text{Struct}(L_C^T)$, there is an assignment of numerical values to the nonzero elements of A so that $U_{i,j}$ is nonzero; this is referred to as an *one-at-time* result [GN93]. However, $\text{Struct}(L_C)$ is *not* a tight bound on $\text{Struct}(L)$. A tight bound on $\text{Struct}(L)$ can be found in [GN87] and [GN93].

Examples:

Some of the properties of sparse matrices, such as zero-free diagonals and reducible/irreducible matrices, depend only on the sparsity structures of the matrices; the actual values of the nonzero elements are irrelevant. As a result, for the examples illustrating such properties, only the sparsity structures will be taken into consideration. The numerical values will not be shown. Nonzero elements will be indicated by $\times$.

1. Following is an example of a reducible matrix:

$$A = \begin{bmatrix} \times & \times & \times & \times & \times & & \times & & & \\ & & & & \times & & \times & \times & \times & \\ & \times & \times & & \times & & & & & \\ & \times & & \times & & \times & & \times & \times & \\ & \times & \times & & \times & & & \times & & \\ \times & \times & \times & \times & & & \times & & \times & \times \\ & \times & & & \times & & & \times & & \\ \times & & & & \times & & & \times & \times & \\ & & & & & & & & & \times \end{bmatrix}.$$

The matrix A can be put into a block upper triangular form using the following permutations:

$$\pi_r = [2, 4, 7, 1, 9, 6, 8, 5, 3, 10],$$

$$\pi_c = [8, 6, 4, 7, 1, 3, 9, 5, 2, 10].$$

Here, $\pi_r(i) = j$ means that row i of the permuted matrix comes from row j of the original matrix. Similarly, $\pi_c(i) = j$ means that column i of the permuted matrix comes from column j of the original matrix. When π_r and π_c are applied to the identity matrix (separately), the corresponding permutation matrices are obtained:

$$P_r = \begin{bmatrix} & \times & & & & & & & & \\ & & \times & & & & & & & \\ & & & \times & & & & & & \\ \times & & & & & & & & & \\ & & & & \times & & & & & \\ & & & & & \times & & & & \\ & & & & & & \times & & & \\ & & & & & & & \times & & \\ & & \times & & & & & & & \\ & & & \times & & & & & & \\ & & & & \times & & & & & \\ & & & & & \times & & & & \\ & & & & & & \times & & & \\ & & & & & & & \times & & \\ & & & & & & & & \times & \\ & & & & & & & & & \times \end{bmatrix}, P_c = \begin{bmatrix} & & & & \times & & & & & \\ & & & & & & & & \times & \\ & & & & & & & & & \times \\ & & \times & & & & & & & \\ & & & \times & & & & & & \\ & & & & \times & & & & & \\ & & & & & \times & & & & \\ & & & & & & \times & & & \\ & & & & & & & \times & & \\ \times & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \\ & & & & & & & & & \times \end{bmatrix}.$$

The permuted matrix $P_r A P_c$ is shown below:

$$P_r A P_c = \begin{bmatrix} \times & & & \times & & & \times & \times & & \\ & \times & \times & & & & \times & & \times & \times \\ & & \times & \times & \times & & \times & \times & & \times \\ & & \times & \times & \times & & \times & & \times & \times \\ & & & & \times & & \times & \times & & \times \\ & & & & & \times & & \times & \times & \times \\ & & & & & \times & \times & \times & \times & \times \\ & & & & & \times & & \times & \times & \times \\ & & & & & & & & & \times \end{bmatrix}.$$

The block triangular form has four 1×1 blocks, one 2×2 block, and one 4×4 block on the diagonal.

2. The advantage of the block triangular form is that only the diagonal blocks have to be factored in the solution of a linear system. As an example, suppose that A is lower block triangular:

$$A = \begin{bmatrix} A_{1,1} & & & \\ A_{2,1} & A_{2,2} & & \\ \vdots & \vdots & \ddots & \\ A_{m,1} & A_{m,2} & \dots & A_{m,m} \end{bmatrix}.$$

Here, m is the number of blocks on the diagonal of A . Consider the solution of the linear system $A\mathbf{x} = \mathbf{b}$. Suppose that $\mathbf{b}$ and $\mathbf{x}$ are partitioned according to the block structure of A :

$$\mathbf{b} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \vdots \\ \mathbf{b}_m \end{bmatrix} \quad \text{and} \quad \mathbf{x} = \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_m \end{bmatrix}.$$

Then the solution can be obtained using a block substitution scheme:

$$A_{k,k}\mathbf{x}_k = \mathbf{b}_k - \sum_{i=1}^{k-1} A_{k,i}\mathbf{x}_i, \quad \text{for } 1 \leq k \leq m.$$

Note that only the diagonal blocks $A_{k,k}$, $1 \leq k \leq m$ have to be factored; Gaussian elimination does not have to be applied to the entire matrix A . This can result in substantial savings in storage and operations.

3. In this example, the matrix A has zero elements on the diagonal:

$$A = \begin{bmatrix} & & & \times & & \times & \times & & \times \\ & & & \times & & & & & \times \\ & & & \times & \times & & \times & & \times \\ & & \times & & \times & & \times & & \times \\ \times & & & & & \times & \times & \times & \times \\ \times & & & & & \times & & \times & \times \\ & \times & \times & & & & \times & \times & \times \\ & \times & \times & \times & \times & \times & & & \\ & & & & \times & & \times & & \\ \times & \times & & & \times & & & & \times \end{bmatrix}.$$

The following permutation matrix P , when applied to the columns of A , will produce a matrix with a zero-free diagonal:

$$P = \begin{bmatrix} & & & & & & & \times \\ & & & \times & & & & \\ & \times & & & & \times & & \\ & & \times & & & & & \\ \times & & & & & & \times & \\ & \times & & & & & & \\ & & \times & & & & & \\ & & & \times & & & & \\ & & & & \times & & & \\ & & & & & \times & & \end{bmatrix}.$$

The sparsity structure of the permuted matrix is shown below:

$$AP = \begin{bmatrix} \times & & \times & \times & & \times & & \\ & \times & & & & \times & & \\ & \times & \times & \times & & \times & & \\ \times & & & \times & \times & & \times & \\ \times & & \times & & \times & \times & \times & \times \\ \times & & & \times & \times & \times & & \\ & \times & \times & & \times & & \times & \times \\ \times & \times & \times & \times & & & \times & \times \\ & & & & \times & & & \times \\ & & & & & \times & \times & \times & \times \end{bmatrix}.$$

The permutation matrix P is obtained by applying the following permutation π to the columns of the identity matrix:

$$\pi = [7, 4, 8, 5, 9, 2, 10, 3, 6, 1].$$

Again, $\pi(i) = j$ means that column i of the permuted matrix comes from column j of the original matrix.

4. Consider the following matrix:

$$A = \begin{bmatrix} 10 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 10 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 10 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 10 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 10 \end{bmatrix},$$

which is diagonal dominant. The matrix can be factored without pivoting for stability. The triangular factors are given below:

$$L = \begin{bmatrix} 1 & & & & & & \\ \frac{1}{10} & 1 & & & & & \\ 0 & 0 & 1 & & & & \\ 0 & 0 & \frac{1}{10} & 1 & & & \\ 0 & \frac{1}{10} & \frac{1}{1000} & 0 & 1 & & \\ \frac{1}{10} & 0 & -\frac{1}{100} & 0 & -\frac{10}{991} & 1 & \\ 0 & 0 & 0 & 0 & \frac{100}{991} & \frac{1}{99199} & 1 \end{bmatrix}, U = \begin{bmatrix} 10 & 0 & 1 & 0 & 1 & 0 & 0 \\ & 10 & -\frac{1}{10} & 0 & \frac{9}{10} & 0 & 0 \\ & & 10 & 0 & 0 & 1 & 0 \\ & & & 10 & 0 & -\frac{1}{10} & 1 \\ & & & & \frac{991}{100} & -\frac{1}{1000} & 1 \\ & & & & & \frac{99199}{9910} & \frac{10}{991} \\ & & & & & & \frac{981980}{99199} \end{bmatrix}.$$

Note that A has 18 nonzero elements, whereas $L + U$ has 26 nonzero elements, showing that there are more nonzero elements in the fill matrix than in A .

5. Consider the matrix A in the previous example. Suppose $\hat{A} = [\hat{a}_{i,j}]$ is obtained from A by swapping the $(1, 1)$ and $(2, 1)$ elements:

$$\hat{A} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 10 & 10 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 10 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 10 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 10 \end{bmatrix}.$$

When Gaussian elimination with partial pivoting is applied to $\hat{A}$, rows 1 and 2 will be interchanged at step 1 of the elimination since $|\hat{a}_{2,1}| > |\hat{a}_{1,1}|$. It can be shown that no more interchanges are needed in the subsequent steps. The triangular factors are given by

$$\hat{L} = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{10} & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{10} & 1 & 0 & 0 & 0 \\ 0 & -1 & \frac{1}{10} & 0 & 1 & 0 & 0 \\ \frac{1}{10} & 1 & -\frac{1}{10} & 0 & -\frac{10}{109} & 1 & 0 \\ 0 & 0 & 0 & 0 & \frac{10}{109} & \frac{10}{10999} & 1 \end{bmatrix}, \hat{U} = \begin{bmatrix} 10 & 10 & 0 & 0 & 1 & 0 & 0 \\ 0 & -1 & 1 & 0 & \frac{9}{10} & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 10 & 0 & -\frac{1}{10} & 1 \\ 0 & 0 & 0 & 0 & \frac{109}{10} & -\frac{1}{10} & 1 \\ 0 & 0 & 0 & 0 & 0 & \frac{10999}{1090} & \frac{10}{109} \\ 0 & 0 & 0 & 0 & 0 & 0 & \frac{108980}{10999} \end{bmatrix}.$$

Even though A (in the previous example) and $\hat{A}$ have the same sparsity structures, their fill matrices have different numbers of nonzero elements. For $\hat{A}$, there are 27 nonzero elements in the fill matrix of $\hat{A}$. While this example is small, it illustrates that the occurrence of fill elements in Gaussian elimination generally depends on both the sparsity structure and the values of the nonzero elements of the matrix.

6. Consider the example in the last two examples again. Note that A has a zero-free diagonal. Both A and $\hat{A}$ have the same sparsity structure. The sparsity structure of $A^T A$ is the same as that of $\hat{A}^T \hat{A}$:

$$A^T A = \begin{bmatrix} \times & \times & \times & & \times & \times & \\ \times & \times & & & \times & & \times \\ \times & & \times & \times & \times & \times & \times \\ & & \times & \times & & & \times \\ \times & \times & \times & & \times & & \times \\ \times & & \times & & & \times & \\ & \times & \times & \times & \times & & \times \end{bmatrix}.$$

(Again, $\times$ denotes a nonzero element.) The Cholesky factor L_C of the symmetric positive definite matrix $A^T A$ has the form:

$$L_C = \begin{bmatrix} \times & & & & & & \\ \times & \times & & & & & \\ \times & \times & \times & & & & \\ & & \times & \times & & & \\ \times & \times & \times & \times & \times & & \\ \times & \times & \times & \times & \times & \times & \\ & \times & \times & \times & \times & \times & \times \end{bmatrix}.$$

Note that $\text{Struct}(L) \neq \text{Struct}(\hat{L})$, but $\text{Struct}(L) \subset \text{Struct}(L_C)$ and $\text{Struct}(\hat{L}) \subset \text{Struct}(L_C)$. Similarly, $\text{Struct}(U) \neq \text{Struct}(\hat{U})$, but $\text{Struct}(U) \subset \text{Struct}(L_C^T)$ and $\text{Struct}(\hat{U}) \subset \text{Struct}(L_C^T)$. This example illustrates that when A has a zero-free diagonal the sparsity structure of the Cholesky factor of $A^T A$ indeed contains the sparsity structure of L (or U^T), irrespective of the choice of the pivot sequence.

7. It is shown in an earlier example that fill elements do occur in Gaussian elimination of sparse matrices. In order to allow for these fill elements during Gaussian elimination, the simple storage schemes (CCS or CRS) may not be sufficient. More sophisticated storage schemes are often needed. The choice of a storage scheme depends on the choice of the factorization algorithm. There are many implementations of sparse Gaussian elimination, such as *profile methods*, *left-looking methods*, *right-looking methods*, and *frontal/multifrontal methods* [DER89], [GL81]. Describing these implementations is beyond the scope of this chapter. Following is a list of pointers to some of the implementations.
 - (a) Factorization of sparse symmetric positive definite matrices: [NP93], [RG91].
 - (b) Factorization of sparse symmetric indefinite matrices: [AGL98], [DR83], [DR95], [Duf04], [Liu87a], [Liu87b].
 - (c) Factorization of sparse nonsymmetric matrices: [DEG⁺99], [Duf77], [DR96], [GP88].

40.4 Modeling and Analyzing Fill

A key component of sparse Gaussian elimination is the exploitation of the sparsity structure of the given matrix and its triangular factors. Graphs are useful in understanding and analyzing how fill elements are introduced in sparse Gaussian elimination. Some basic graph-theoretical tools and results are described in this section. Others can be found in [EL92], [GL81], [GL93], [GNP94], [Liu86], [Liu90], and [Sch82].

In this and the next sections, all graphs (bipartite graphs, directed graphs, and undirected graphs) are simple. That is, loops and multiple edges are not allowed. More information on graphs can be found in Chapter 28 (graphs), Chapter 29 (digraphs), and Chapter 30 (bipartite graphs).

Definitions:

Consider a sparse nonsymmetric matrix $A = [a_{ij}]$. Let $R = \{r_1, r_2, \dots, r_n\}$ be a set of “row” vertices associated with the rows of A . Similarly, let $C = \{c_1, c_2, \dots, c_n\}$ be a set of “column” vertices associated with the columns of A . The **bipartite graph** or **bigraph** of A , denoted by $H(A) = (R, C, E)$, can be associated with the sparsity structure of A . There is an edge $\{r_i, c_j\} \in E$ if and only $a_{ij} \neq 0$.

Let $A = [a_{ij}]$ be a sparse nonsymmetric matrix. Suppose that A has a zero-free diagonal, and assume that the pivots are always chosen from the diagonal. Then the sparsity structure of A can be represented by a **directed graph** or **digraph**, denoted by $\Gamma(A) = (X, E)$. Here, $X = \{x_1, x_2, \dots, x_n\}$, with x_i , $1 \leq i \leq n$, representing column i and row i of A . There is a directed edge or arc $(x_i, x_j) \in E$ if and only if $a_{ij} \neq 0$, for $i \neq j$. (The nonzero elements on the diagonal of A are not represented.)

Suppose $A = [a_{ij}]$ is symmetric and positive definite, and assume that the pivots are always chosen from the diagonal. Then an **undirected graph** or **graph** (when the context is clear) $G(A) = (X, E)$ can be used to represent the sparsity structure of A . Let $X = \{x_1, x_2, \dots, x_n\}$, with x_i representing row i and column i of A . There is an (undirected) edge $\{x_i, x_j\} \in E$ if and only if $a_{ij} \neq 0$ (and, hence, $a_{ji} \neq 0$), for $i \neq j$. (The nonzero elements on the diagonal of A are not represented.)

A **path** in a graph (which can be a bigraph, digraph, or undirected graph) is a sequence of *distinct* vertices $(x_{s_1}, x_{s_2}, \dots, x_{s_t})$ such that there is an edge between every pair of consecutive vertices. For bigraphs and undirected graphs, $\{x_{s_p}, x_{s_{p+1}}\}$, $1 \leq p \leq t-1$, is an (undirected) edge. For digraphs, the path is a directed path, and $(x_{s_p}, x_{s_{p+1}})$, $1 \leq p \leq t-1$, is an arc.

Let A be an n by n matrix. After k ($1 \leq k \leq n$) steps of Gaussian elimination, the matrix remaining to be factored is the trailing submatrix that consists of the elements in the last $(n-k)$ rows and the last

$(n - k)$ columns of A . The graph (bipartite, digraph, or undirected graph) associated with this $(n - k)$ by $(n - k)$ trailing matrix is the k -th **elimination graph**.

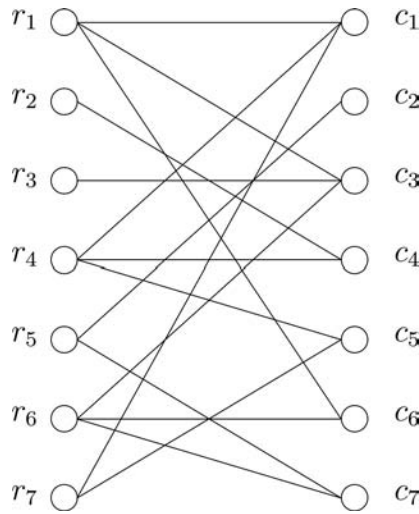
Facts:

1. [PM83] Let $H^{(0)}, H^{(1)}, H^{(2)}, \dots, H^{(n)}$ be the sequence of elimination bigraphs associated with the Gaussian elimination of a sparse nonsymmetric matrix $A = [a_{i,j}]$. The initial bigraph $H^{(0)}$ is the bigraph of A . Each elimination bigraph can be obtained from the previous one through a simple transformation. Suppose that the nonzero element $a_{s,t}$ is chosen as the pivot at step k , $1 \leq k \leq n$. Then the edge corresponding to $a_{s,t}$, $\{r_s, c_t\}$, is removed from $H^{(k-1)}$, together with all the edges incident to r_s and c_t . To obtain the next elimination bigraph $H^{(k)}$ from the modified $H^{(k-1)}$, an edge $\{r, c\}$ is added if there is a path (r, c_t, r_s, c) in the original $H^{(k-1)}$ and if $\{r, c\}$ is not already in $H^{(k-1)}$. The new edges added to create $H^{(k)}$ correspond to the fill elements introduced when $a_{s,t}$ is used to eliminate row s and column t . The bigraph $H^{(k)}$ represents the sparsity structure of the matrix remaining to be factored after row s and column t are eliminated.
2. [RT78] Let A be a sparse nonsymmetric matrix. Suppose that A has a zero-free diagonal and assume that pivots are restricted to the diagonal. Then Gaussian elimination can be modeled using elimination digraphs. Let $\Gamma^{(0)}, \Gamma^{(1)}, \Gamma^{(2)}, \dots, \Gamma^{(n)}$ be the sequence of elimination digraphs. The initial digraph $\Gamma^{(0)}$ is the digraph of A . Each elimination digraph can be obtained from the previous one through a simple transformation. Without loss of generality, assume that Gaussian elimination proceeds from row/column 1 to row/column n . Consider step k , $1 \leq k \leq n$. Vertex x_k is removed from $\Gamma^{(k-1)}$, together with all the arcs of the form (x_k, u) and (v, x_k) , where u and v are vertices in $\Gamma^{(k-1)}$. To obtain the next elimination digraph $\Gamma^{(k)}$ from the modified $\Gamma^{(k-1)}$, an arc (r, c) is added if there is a directed path (r, x_k, c) in the original $\Gamma^{(k-1)}$ and if (r, c) is not already in $\Gamma^{(k-1)}$. The new arcs added to create $\Gamma^{(k)}$ correspond to the fill elements introduced by Gaussian elimination at step k . The digraph $\Gamma^{(k)}$ represents the sparsity structure of the matrix remaining to be factored after k steps of Gaussian elimination.
3. [Ros72] For a symmetric positive definite matrix A , the elimination graphs associated with Gaussian elimination can be represented by (undirected) graphs. Let the elimination graphs be denoted by $G^{(0)}, G^{(1)}, G^{(2)}, \dots$, and $G^{(n)}$. The initial graph $G^{(0)}$ is the graph of A . Each elimination graph can be obtained from the previous one through a simple transformation. Without loss of generality, assume that Gaussian elimination proceeds from row/column 1 to row/column n . Consider step k , $1 \leq k \leq n$. Vertex x_k is removed from $G^{(k-1)}$, together with all its incident edges. To obtain the next elimination graph $G^{(k)}$ from the modified $G^{(k-1)}$, an (undirected) edge $\{r, c\}$ is added if there is a path (r, x_k, c) in the original $G^{(k-1)}$ and if $\{r, c\}$ is not already in $G^{(k-1)}$. The new edges added to create $G^{(k)}$ correspond to the fill elements introduced by Gaussian elimination at step k . The graph $G^{(k)}$ represents the sparsity structure of the matrix remaining to be factored after k steps of Gaussian elimination.
4. [GL80b], [RTL76] Consider an n by n symmetric positive definite matrix A . Let $G = (X, E)$ be the (undirected) graph of A and let $X = \{x_1, x_2, \dots, x_n\}$. Denote the Cholesky factor of A by L . For $i > j$, the (i, j) element of L is nonzero if and only there is a path $(x_i, x_{s_1}, x_{s_2}, \dots, x_{s_t}, x_j)$ in G such that $s_p < j (< i)$, for $1 \leq p \leq t$. Such a path is sometimes referred to as a *fill path*. Note that t can be zero, which corresponds to $\{x_i, x_j\} \in E$.

Examples:

1. As one may observe, the operations involved in the generation of the elimination bigraphs, digraphs, and graphs are very similar. Thus, only one example will be illustrated here. Consider the nonsymmetric matrix $A = [a_{i,j}]$ in Figure 40.3. The bigraph H of A is depicted in Figure 40.4. Suppose that $a_{1,1}$ is used to elimination row 1 and column 1. After the first step of Gaussian elimination, the remaining matrix is shown in Figure 40.5. Following the recipe given above, the edges $\{r_1, c_1\}$, $\{r_1, c_3\}$, $\{r_1, c_6\}$, $\{r_4, c_1\}$, and $\{r_7, c_1\}$ are to be removed from the bigraph H in Figure 40.4. Then r_1

$$A = \begin{bmatrix} \times & & \times & & \times & & \\ & & & \times & & & \\ & & & & \times & & \\ \times & & & & & \times & \times \\ & \times & & & & & \times \\ & & \times & & & \times & \times \\ \times & & & & \times & & \end{bmatrix}.$$

FIGURE 40.3 A sparse nonsymmetric matrix A .FIGURE 40.4 The bigraph of the matrix A in Figure 40.3.

$$A' = \begin{bmatrix} & & \times & & & & \\ & \times & & & & & \\ & + & \times & \times & + & & \\ \times & & & & & & \times \\ & \times & & & \times & \times & \\ & + & & \times & + & & \end{bmatrix}.$$

FIGURE 40.5 The remaining matrix after the first step of Gaussian elimination on the matrix A in Figure 40.3.

and c_1 are also removed from the bigraph H . The new bigraph is obtained by adding to H the following edges:

- (a) $\{r_4, c_3\}$ (because of the path (r_4, c_1, r_1, c_3) in the original H).
- (b) $\{r_4, c_6\}$ (because of the path (r_4, c_1, r_1, c_6) in the original H).
- (c) $\{r_7, c_3\}$ (because of the path (r_7, c_1, r_1, c_3) in the original H).
- (d) $\{r_7, c_6\}$ (because of the path (r_7, c_1, r_1, c_6) in the original H).

The new bigraph is shown in Figure 40.6, in which the new edges are shown as dashed lines. Note that the new bigraph is exactly the bigraph of A' .

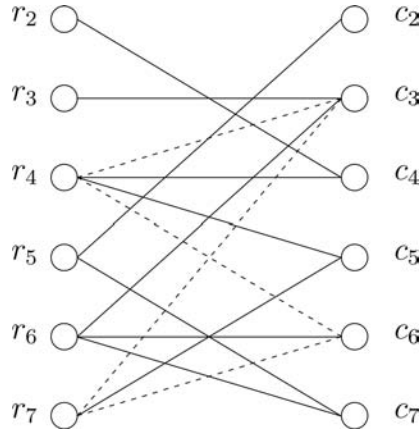


FIGURE 40.6 The bigraph of the matrix A' in Figure 40.5.

2. Let $A = [a_{i,j}]$ be a symmetric and positive definite matrix:

$$A = \begin{bmatrix} \times & & \times & & & & \times \\ & \times & \times & & \times & & \\ \times & \times & \times & & & & \\ & & & \times & \times & & \times \\ & \times & & \times & \times & \times & \\ & & & & \times & \times & \times \\ & & & \times & & \times & \\ \times & & & & & \times & \times \end{bmatrix}.$$

Assume that exact numerical cancellations do not occur. Then the sparsity structure of the Cholesky factor $L = [\ell_{i,j}]$ of A is given below:

$$L = \begin{bmatrix} \times & & & & & & \\ & \times & & & & & \\ \times & \times & \times & & & & \\ & & & \times & & & \\ & \times & + & \times & \times & & \\ & & & & \times & \times & \\ & & & \times & + & + & \times \\ \times & & + & & + & \times & + & \times \end{bmatrix}.$$

The symbol $+$ represents a fill element. The (undirected) graph G of A is shown in Figure 40.7. There are several fill paths in G . The fill path $(x_5, x_2, x_3, x_1, x_8)$ corresponds to the fill element $\ell_{8,5}$ in L . Another example is the fill path (x_7, x_4, x_5, x_6) , which corresponds to the fill element $\ell_{7,6}$ in L .

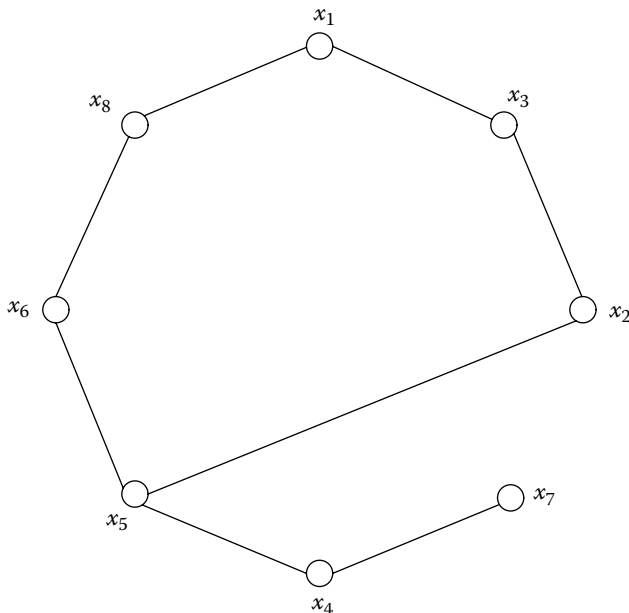


FIGURE 40.7 An example illustrating fill paths.

40.5 Effect of Reorderings

Let A be a sparse nonsingular matrix. As noted above, the occurrence of fill elements in Gaussian elimination generally depends on the values of the nonzero elements in A (which affect the choice of pivots if numerical stability is a concern) and the sparsity structure of A . This section will consider some techniques that will help preserve the sparsity structure in the factorization of A .

Definitions:

Let A be a sparse matrix. Suppose that G is a graph associated with A as described in the previous section; the graph can be a bipartite graph, directed graph, or undirected graph, depending on whether A is nonsymmetric or symmetric and whether the pivots are chosen to be on the diagonal. The **fill graph** G^F of A is G , together with all the additional edges corresponding to the fill elements that occur during Gaussian elimination.

An **elimination ordering** (or **elimination sequence**) for the rows (or columns) of a matrix is a bijection $\alpha : \{1, 2, \dots, n\} \rightarrow \{1, 2, \dots, n\}$. It specifies the order in which the rows (or columns) of the matrix are eliminated during Gaussian elimination.

A **perfect elimination ordering** is an elimination ordering that does not produce any fill elements during Gaussian elimination.

Consider an n by n sparse matrix A . Let f_i and ℓ_i be the column indices of the *first* and *last* nonzero elements in row i of A , respectively. The **envelope** of A is the set

$$\{(i, j) : f_i \leq j \leq \ell_i, \text{ for } 1 \leq i \leq n\}.$$

That is, all the elements between the first and last nonzero elements of every row are in the envelope. The **profile** of a matrix is the number of elements in the envelope.

Facts:

The problem of reordering a sparse matrix is combinatorial in nature. The facts stated below are some of fundamental ones. Others can be found, for example, in [GL81] and [Gol04].

1. [GL81] An elimination ordering for the rows/columns of the matrix corresponds to a permutation of the rows/columns.
2. [DER89], [GL81] The choice of an elimination ordering will affect the number of fill elements in the triangular factors.
3. [GL81], [Ros72] When A is sparse symmetric positive definite, an elimination ordering α can be determined by analyzing the sparsity structure of A . Equivalently, α can be obtained by analyzing the sequence of elimination graphs. The elimination ordering provides a symmetric permutation of A . Let P denote the permutation matrix corresponding to α . Let G^F be the fill graph of the matrix PAP^T . There exists a perfect elimination ordering for G^F and G^F is a chordal graph.
4. [Yan81] For sparse symmetric positive definite matrices, finding the optimal elimination ordering (i.e., an elimination ordering that minimizes the number of fill elements in the Cholesky factor) is NP-complete. This implies that almost all reordering techniques are heuristic in nature.
5. [DER89] When A is a sparse nonsymmetric matrix, the elimination orderings for the rows and columns of A have to be chosen to preserve the sparsity structure and to maintain numerical stability.
6. [GN85] Suppose that A is a sparse nonsymmetric matrix. If partial pivoting (by rows) is used to maintain numerical stability, then an elimination ordering for the columns of A can be chosen to preserve the sparsity structure.

Examples:

1. Consider the following two diagonally dominant matrices:

$$A = \begin{bmatrix} 7 & 1 & -1 & 1 & -1 \\ -1 & 7 & 0 & 0 & 0 \\ 1 & 0 & 7 & 0 & 0 \\ -1 & 0 & 0 & 7 & 0 \\ 1 & 0 & 0 & 0 & 7 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 7 & 0 & 0 & 0 & 1 \\ 0 & 7 & 0 & 0 & -1 \\ 0 & 0 & 7 & 0 & 1 \\ 0 & 0 & 0 & 7 & -1 \\ -1 & 1 & -1 & 1 & 7 \end{bmatrix}.$$

Applying Gaussian elimination to A produces the following triangular factors:

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -\frac{1}{7} & 1 & 0 & 0 & 0 \\ \frac{1}{7} & -\frac{1}{50} & 1 & 0 & 0 \\ -\frac{1}{7} & \frac{1}{50} & -\frac{1}{51} & 1 & 0 \\ \frac{1}{7} & -\frac{1}{50} & \frac{1}{51} & -\frac{1}{52} & 0 \end{bmatrix} \begin{bmatrix} 7 & 1 & -1 & 1 & -1 \\ 0 & \frac{50}{7} & -\frac{1}{7} & \frac{1}{7} & -\frac{1}{7} \\ 0 & 0 & \frac{357}{50} & -\frac{7}{50} & \frac{7}{50} \\ 0 & 0 & 0 & \frac{364}{51} & -\frac{7}{51} \\ 0 & 0 & 0 & 0 & \frac{371}{52} \end{bmatrix}.$$

Applying Gaussian elimination to B , on the other hand, produces the following triangular factors:

$$B = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ -\frac{1}{7} & \frac{1}{7} & -\frac{1}{7} & \frac{1}{7} & 1 \end{bmatrix} \begin{bmatrix} 7 & 0 & 0 & 0 & 1 \\ 0 & 7 & 0 & 0 & -1 \\ 0 & 0 & 7 & 0 & 1 \\ 0 & 0 & 0 & 7 & -1 \\ 0 & 0 & 0 & 0 & \frac{53}{7} \end{bmatrix}.$$

The two matrices A and B have the *same* numbers of nonzero elements, but their respective triangular factors have *very different* numbers of fill elements. In fact, the triangular factors of B

have *no* fill elements. Note that B can be obtained by permuting the rows and columns of A . Let P be the following permutation matrix:

$$P = \begin{bmatrix} 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

Then $B = PAP^T$; that is, B is obtained by *reversing* the order of the rows and columns. This example illustrates that permuting the rows and columns of a sparse matrix may have a drastic effect on the sparsity structures of the triangular factors in Gaussian elimination.

2. A popular way to preserve the sparsity structure of a sparse matrix during Gaussian elimination is to find elimination orderings so that the nonzero elements in the permuted matrix are near the diagonal. This can be accomplished, for example, by permuting the matrix so that it has a small envelope or profile. The Cuthill–McKee algorithm [CM69] and the reverse Cuthill–McKee algorithms [Geo71], [LS76] are well-known heuristics for producing reorderings that reduce the profile of a sparse symmetric matrix. The permuted matrix can be factored using the *envelope* or *profile* method [GL81], which is similar to the factorization methods for band matrices. The storage scheme for the profile method is very simple. The permuted matrix is stored by rows. If the symmetric matrix is positive definite, then for every row of the permuted matrix, all the elements between the first nonzero element and the diagonal elements are stored. It is easy to show that the fill elements in the triangular factor can only occur inside the envelope of the lower triangular part of the permuted matrix.
3. Although the profile method is easy to implement, it is not designed to reduce the number of fill elements in Gaussian elimination. The nested dissection algorithm [Geo73], which is based on a divide-and-conquer idea, is a well-known heuristic for preserving the sparsity structure. Let A be a symmetric positive definite matrix, and let $G = (X, E)$ be the (undirected) graph of A . Without loss of generality, assume that G is connected. Let $S \subseteq X$. Suppose that S is removed from G . Also assume that all edges incident to every vertex in S are removed from G . Denote the remaining graph by $G(X - S)$. If S is chosen so that $G(X - S)$ contains one or more disconnected components, then the set S is referred to as a *separator*. Consider the following reordering strategy: renumber the vertices of the disconnected components of $G(X - S)$ first, and renumber of the vertices of S last. Now pick vertex x in one component and vertex y in another component. The renumbering scheme ensures that there is *no* fill path between x and y in G . This is a heuristic way to limit the creation of fill elements. The renumbering corresponds to a symmetric permutation of the rows and columns of the matrix. Consider the example in Figure 40.8. The removal of the vertices in S (together with the incident edges) divides the mesh into two joint meshes (labeled G_1 and G_2). Suppose that the mesh points in G_1 are renumbered before those in G_2 . Then the matrix on the right in Figure 40.8 shows the sparsity structure of the permuted matrix. Note the block structure of the permuted matrix. The blocks A_1 and A_2 correspond to mesh points in G_1 and G_2 , respectively. The block A_S corresponds to the mesh points in S . The nonzero elements in the off-diagonal blocks C_1/C_2 correspond to the edges between G_1/G_2 and S . The unlabeled blocks are entirely zero. This is referred to as the “dissection” strategy. When the strategy is applied recursively to the disconnected components, more zero (but smaller) blocks will be created in A_1 and A_2 . The resulting reordering is called a “nested dissection” ordering. It can be shown that, when the nested dissection algorithm is applied to a k by k mesh (like the example in the Introduction), the number of nonzero elements in the Cholesky factor will be $O(k^2 \log k)$ and the number of operations required to compute the Cholesky factor will be $O(k^3)$. Incidentally, for the k by k mesh, it has been proved that the number of nonzero elements in the Cholesky factor and the number of operations required to compute the triangular factorization are at least $O(k^2 \log k)$ and $O(k^3)$, respectively [HMR73], [Geo73]. Thus, nested

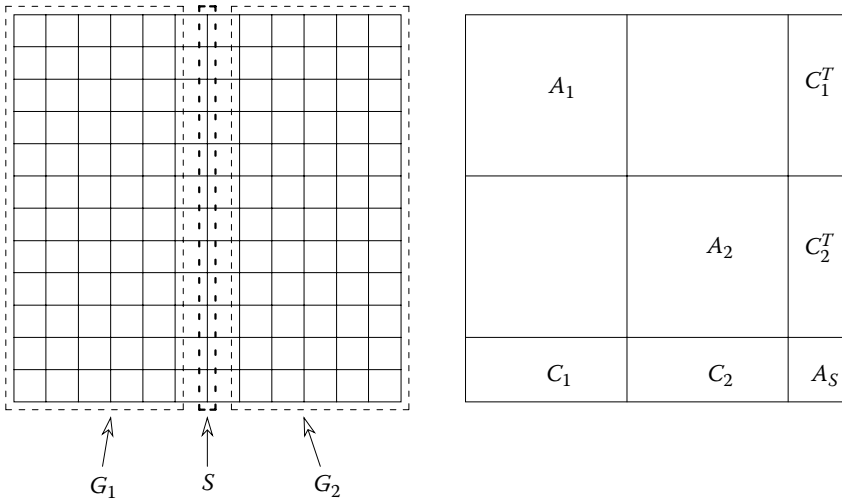


FIGURE 40.8 An example of the dissection strategy.

dissection orderings can be optimal asymptotically. In recent years, higher quality nested dissection orderings have been obtained for general sparse symmetric matrices by using more sophisticated graph partitioning techniques to generate the separators [HR98], [PSL90], [Sch01].

4. The nested dissection algorithm is a “top-down” algorithm since it identifies the vertices (i.e., rows/columns) to be reordered last. The minimum degree algorithm [TW67] is a “bottom-up” algorithm. It is a heuristic that is best described using the elimination graphs. Let $G^{(0)}$ be the (undirected) graph of a sparse symmetric positive definite matrix. The minimum degree algorithm picks the vertex x_m with the *smallest* degree (i.e., the smallest number of incident edges) to be eliminated; that is, x_m is to be reordered as the first vertex. Then x_m , together with all the edges incident to x_m , are eliminated from $G^{(0)}$ to generate the next elimination graph $G^{(1)}$. This process is repeated until all the vertices are eliminated. The order in which the vertices are eliminated is a minimum degree ordering. Note that if several vertices have the minimum degree in the current elimination graph, then ties have to be broken. It is well known that the quality of a minimum degree ordering can be influenced by the choice of the tie-breaking strategy [BS90]. Several efficient implementations of the minimum degree algorithm are available [ADD96], [GL80b], [GL80a], [Liu85]. An excellent survey of the minimum degree algorithm can be found in [GL89].
5. The minimum deficiency algorithm [TW67] is another bottom-up strategy. It is similar to the minimum degree algorithm, except that the vertex whose elimination would introduce the *fewest* fill elements will be eliminated at each step. In general, the minimum deficiency algorithm is much more expensive to implement than the minimum degree algorithm. This is because the former one needs the *look-ahead* to predict the number of fill elements that would be introduced. However, inexpensive approximations to the minimum deficiency algorithms have been proposed recently [NR99], [RE98].
6. For a sparse nonsymmetric matrix $A = [a_{i,j}]$, there are analogs of the minimum degree and minimum deficiency algorithms. The Markowitz scheme [Mar57] is the nonsymmetric version of the minimum degree algorithm for sparse nonsymmetric matrices. Recall that $nnz(A_{i,1:n})$ is the number of nonzero elements in row i of A and $nnz(A_{1:n,j})$ is the number of nonzero elements in column j of A . For each nonzero $a_{i,j}$ in A , define its “Markowitz” cost to be the product $[nnz(A_{i,1:n}) - 1][nnz(A_{1:n,j}) - 1]$, which would be the number of nonzero elements in the rank-1 update if $a_{i,j}$ were chosen as pivot. At each step of Gaussian elimination, the nonzero element that has the *smallest* Markowitz cost will be chosen as the pivot. After the elimination, the Markowitz costs of all the nonzero elements, including the fill elements, are updated to reflect the change in

the sparsity structure before proceeding to the next step. If A is symmetric and positive definite, and if pivots are chosen from the diagonal, then the Markowitz scheme is the same as the minimum degree algorithm.

7. The Markowitz scheme for sparse nonsymmetric matrices attempts to preserve the sparsity structure by minimizing the number of nonzero elements introduced into the triangular factors at each step of Gaussian elimination. The resulting pivots may not lead to a numerically stable factorization. For example, the magnitude of a pivot may be too small compared to the magnitudes of the other nonzero elements in the matrix. To enhance numerical stability, a modified Markowitz scheme is often used. Denote the matrix to be factored by $A = [a_{i,j}]$. Let $s = \max\{|a_{i,j}| : a_{i,j} \neq 0\}$. Let τ be a given tolerance; e.g., τ can be 0.01 or 0.001. Without loss of generality, consider the first step of Gaussian elimination of A . Instead of considering all nonzero elements in A , let $C = \{a_{i,j} : |a_{i,j}| \geq \tau s\}$. Thus, C is the subset of nonzero elements whose magnitudes are larger than or equal to τs ; it is the set of candidate pivots. Then the pivot search is limited to applying the Markowitz scheme to the nonzero elements in C . This is a compromise between preserving sparsity and maintaining numerical stability. The two parameters τ and s are usually fixed throughout the entire Gaussian elimination process. The modified scheme is often referred to as the *threshold pivoting* scheme [Duf77].
8. As noted earlier, if Gaussian elimination with partial pivoting is used to factor a sparse nonsymmetric matrix A , then the columns may be permuted to preserve sparsity. Suppose that A has a zero-free diagonal, and let L_C be the Cholesky factor of the symmetric positive definite matrix $A^T A$. Since $\text{Struct}(L) \subseteq \text{Struct}(L_C)$ and $\text{Struct}(U) \subseteq \text{Struct}(L_C^T)$, a possibility is to make sure that L_C is sparse. In other words, one can choose a permutation P_{L_C} for $A^T A$ to reduce the number of nonzero elements in the Cholesky factor of $P_{L_C}^T (A^T A) P_{L_C}$. Note that $P_{L_C}^T (A^T A) P_{L_C} = (AP_{L_C})^T (AP_{L_C})$. Thus, P_{L_C} can be applied to the columns of A [GN85], [GN87].

Author Note

This work was supported by the Director, Office of Science, U.S. Department of Energy under contract no. DE-AC03-76SF00098.

References

- [ADD96] Patrick R. Amestoy, Timothy A. Davis, and Iain S. Duff. An approximate minimum degree ordering algorithm. *SIAM J. Matrix Anal. Appl.*, 17(4):886–905, 1996.
- [AGL98] Cleve Ashcraft, Roger G. Grimes, and John G. Lewis. Accurate symmetric indefinite linear equation solvers. *SIAM J. Matrix Anal. Appl.*, 20(2):513–561, 1998.
- [BS90] Piotr Berman and Georg Schnitger. On the performance of the minimum degree ordering for Gaussian elimination. *SIAM J. Matrix Anal. Appl.*, 11(1):83–88, 1990.
- [CM69] E. Cuthill and J. McKee. Reducing the bandwidth of sparse symmetric matrices. In *Proceedings of the 24th ACM National Conference*, pp. 157–172. ACM, Aug. 1969.
- [DEG⁺99] James W. Demmel, Stanley C. Eisenstat, John R. Gilbert, Xiaoye S. Li, and Joseph W.H. Liu. A supernodal approach to sparse partial pivoting. *SIAM J. Matrix Anal. Appl.*, 20(3):720–755, 1999.
- [DER89] I.S. Duff, A.M. Erisman, and J.K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, Oxford, 1989.
- [DR83] I.S. Duff and J.K. Reid. The multifrontal solution of indefinite sparse symmetric linear equations. *ACM Trans. Math. Software*, 9(3):302–325, 1983.
- [DR95] I.S. Duff and J.K. Reid. MA47, a Fortran code for direct solution of indefinite sparse symmetric linear systems. Technical Report RAL 95-001, Rutherford Appleton Laboratory, Oxfordshire, U.K., 1995.

- [DR96] I.S. Duff and J.K. Reid. The design of MA48: A code for the direct solution of sparse unsymmetric linear systems of equations. *ACM Trans. Math. Software*, 22(2):187–226, 1996.
- [Duf77] I.S. Duff. MA28 — a set of Fortran subroutines for sparse unsymmetric linear equations. Technical Report AERE R-8730, Harwell, Oxfordshire, U.K., 1977.
- [Duf81] I.S. Duff. On algorithms for obtaining a maximum transversal. *ACM Trans. Math. Software*, 7(3):315–330, 1981.
- [Duf04] I.S. Duff. MA57 — a code for the solution of sparse symmetric definite and indefinite systems. *ACM Trans. Math. Software*, 30(2):118–144, 2004.
- [DW88] I. S. Duff and Torbjörn Wiberg. Remarks on implementation of $O(n^{\frac{1}{2}}\tau)$ assignment algorithms. *ACM Trans. Math. Software*, 14(3):267–287, 1988.
- [EL92] Stanley C. Eisenstat and Joseph W.H. Liu. Exploiting structural symmetry in unsymmetric sparse symbolic factorization. *SIAM J. Matrix Anal. Appl.*, 13(1):202–211, 1992.
- [Geo71] John Alan George. *Computer Implementation of the Finite Element Method*. Ph.D. thesis, Dept. of Computer Science, Stanford University, CA, 1971.
- [Geo73] Alan George. Nested dissection of a regular finite element mesh. *SIAM J. Numer. Anal.*, 10(2):345–363, 1973.
- [GL80a] Alan George and Joseph W.H. Liu. A fast implementation of the minimum degree algorithm using quotient graphs. *ACM Trans. Math. Software*, 6(3):337–358, 1980.
- [GL80b] Alan George and Joseph W.H. Liu. A minimal storage implementation of the minimum degree algorithm. *SIAM J. Numer. Anal.*, 17(2):282–299, 1980.
- [GL81] Alan George and Joseph W.H. Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall, Upper Saddle River, NJ, 1981.
- [GL89] Alan George and Joseph W.H. Liu. The evolution of the minimum degree ordering algorithm. *SIAM Review*, 31(1):1–19, 1989.
- [GL93] John R. Gilbert and Joseph W.H. Liu. Elimination structures for unsymmetric sparse lu factors. *SIAM J. Matrix Anal. Appl.*, 14(2):334–352, 1993.
- [GN85] Alan George and Esmond Ng. An implementation of Gaussian elimination with partial pivoting for sparse systems. *SIAM J. Sci. Stat. Comput.*, 6(2):390–409, 1985.
- [GN87] Alan George and Esmond Ng. Symbolic factorization for sparse Gaussian elimination with partial pivoting. *SIAM J. Sci. Stat. Comput.*, 8(6):877–898, 1987.
- [GN93] John R. Gilbert and Esmond G. Ng. Predicting structure in nonsymmetric sparse matrix factorizations. In Alan George, John R. Gilbert, and Joseph W.H. Liu, Eds., *Graph Theory and Sparse Matrix Computation*, vol. IMA #56, pp. 107–140. Springer-Verlag, Heidelberg, 1993.
- [GNP94] John R. Gilbert, Esmond G. Ng, and Barry W. Peyton. An efficient algorithm to compute row and column counts for sparse Cholesky factorization. *SIAM J. Matrix Anal. Appl.*, 15(4):1075–1091, 1994.
- [Gol04] M.C. Golumbic. *Algorithmic Graph Theory and Perfect Graphs*, vol. 57 *Annals of Discrete Mathematics*. North Holland, NY, 2nd ed., 2004.
- [GP88] John R. Gilbert and Tim Peierls. Sparse partial pivoting in time proportional to arithmetic operations. *SIAM J. Sci. Stat. Comput.*, 9(5):862–874, 1988.
- [HMR73] Alan J. Hoffman, Michael S. Martin, and Donald J. Rose. Complexity bounds for regular finite difference and finite element grids. *SIAM J. Numer. Anal.*, 10(2):364–369, 1973.
- [HR98] Bruce Hendrickson and Edward Rothberg. Improving the run time and quality of nested dissection ordering. *SIAM J. Sci. Comput.*, 20(2):468–489, 1998.
- [Liu85] Joseph W.H. Liu. Modification of the minimum-degree algorithm by multiple elimination. *ACM Trans. Math. Software*, 11(2):141–153, 1985.
- [Liu86] Joseph W. Liu. A compact row storage scheme for Cholesky factors using elimination trees. *ACM Trans. Math. Software*, 12(2):127–148, 1986.
- [Liu87a] Joseph W.H. Liu. On threshold pivoting in the multifrontal method for sparse indefinite systems. *ACM Trans. Math. Software*, 13(3):250–261, 1987.